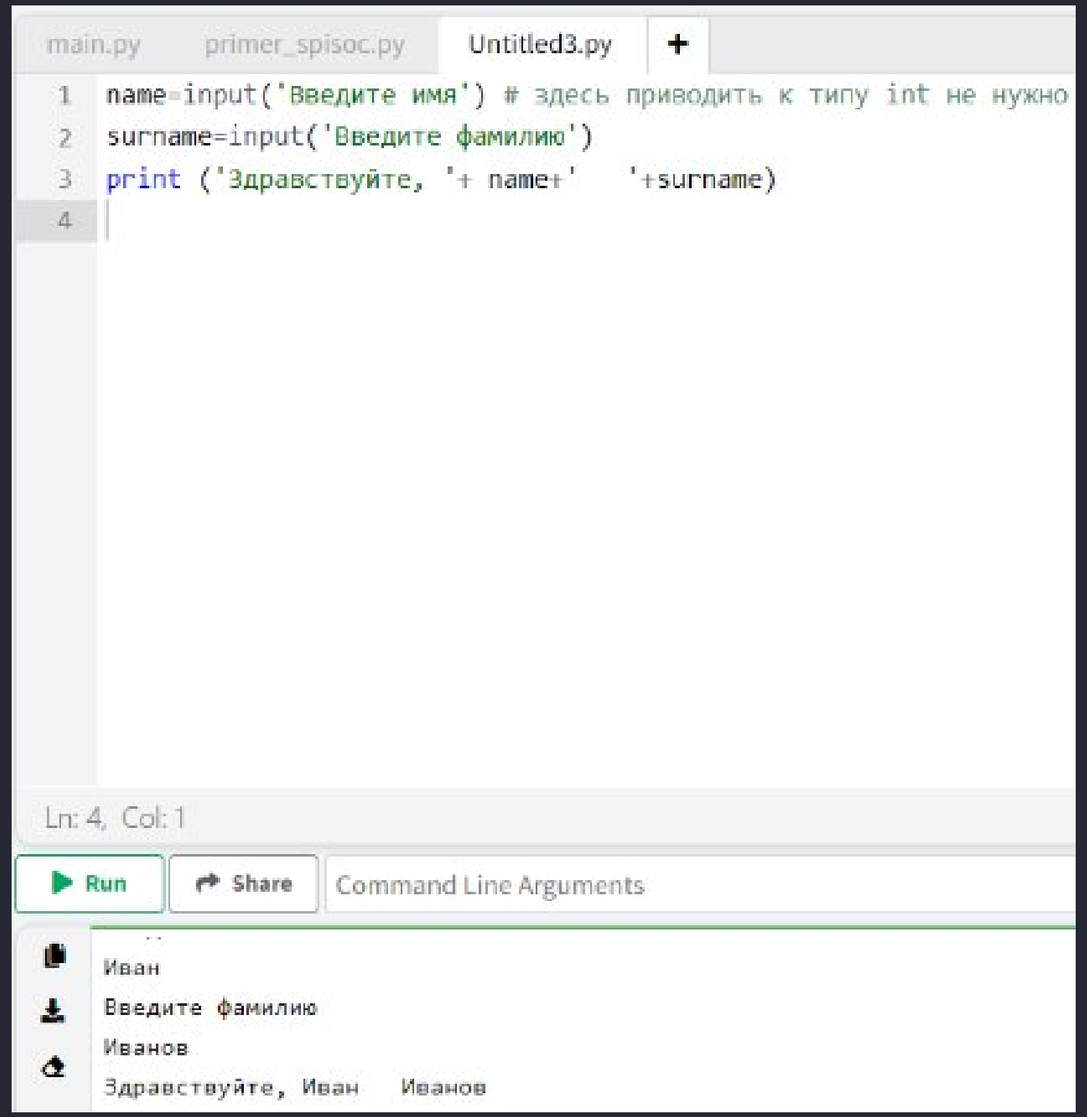
Конкатенация

```
name = input('Введите имя’)
surname = input('Введите фамилию')
print ('Здравствуйте, '+ name+'   '+surname)
```

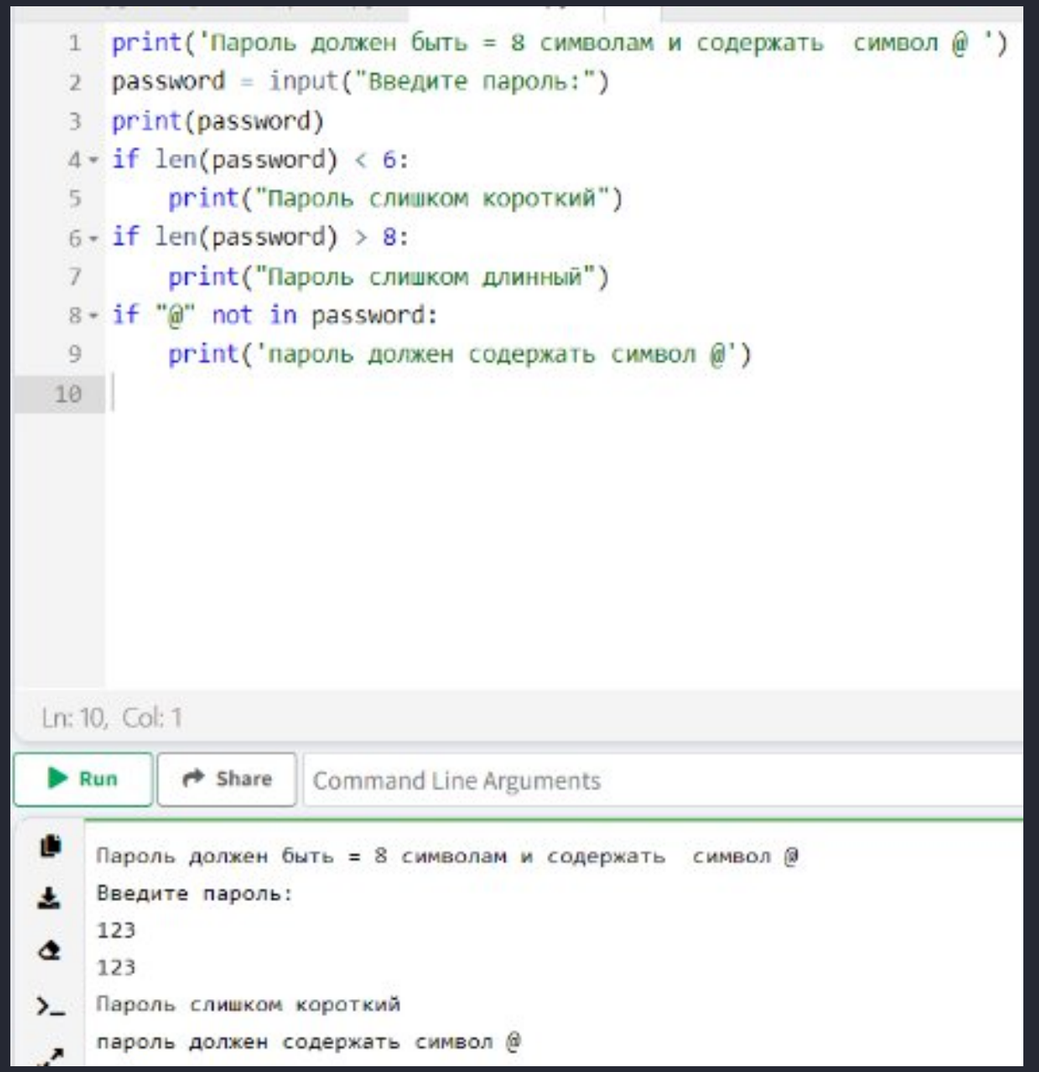
```
#Вывод
Введите имя
Иван
Введите фамилию
Иванов
Здравствуйте, Иван Иванов
```



Строки можно **соединять** с помощью оператора **+**, создавая новую строку, состоящую из двух или более исходных строк.

Функция len()

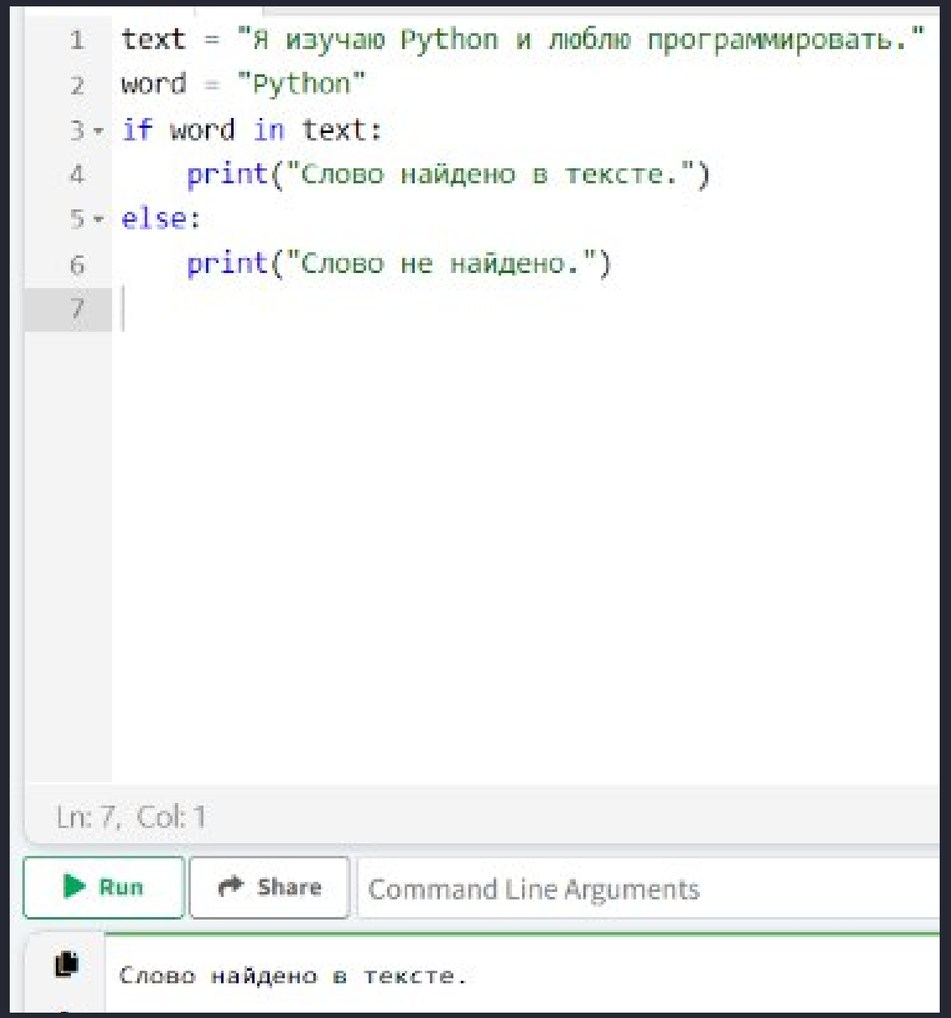
```
print('Пароль должен быть = 8 символам и содержать  символ @ ')
password = input("Введите пароль:")
print(password)
if len(password) < 6:
    print("Пароль слишком короткий")
if len(password) > 8:
    print("Пароль слишком длинный")
if "@" not in password:
    print('пароль должен содержать символ @')
```



Функция **len()** возвращает количество элементов в списке. Она очень полезна для определения размера списка и итерации по его элементам.

Оператор in

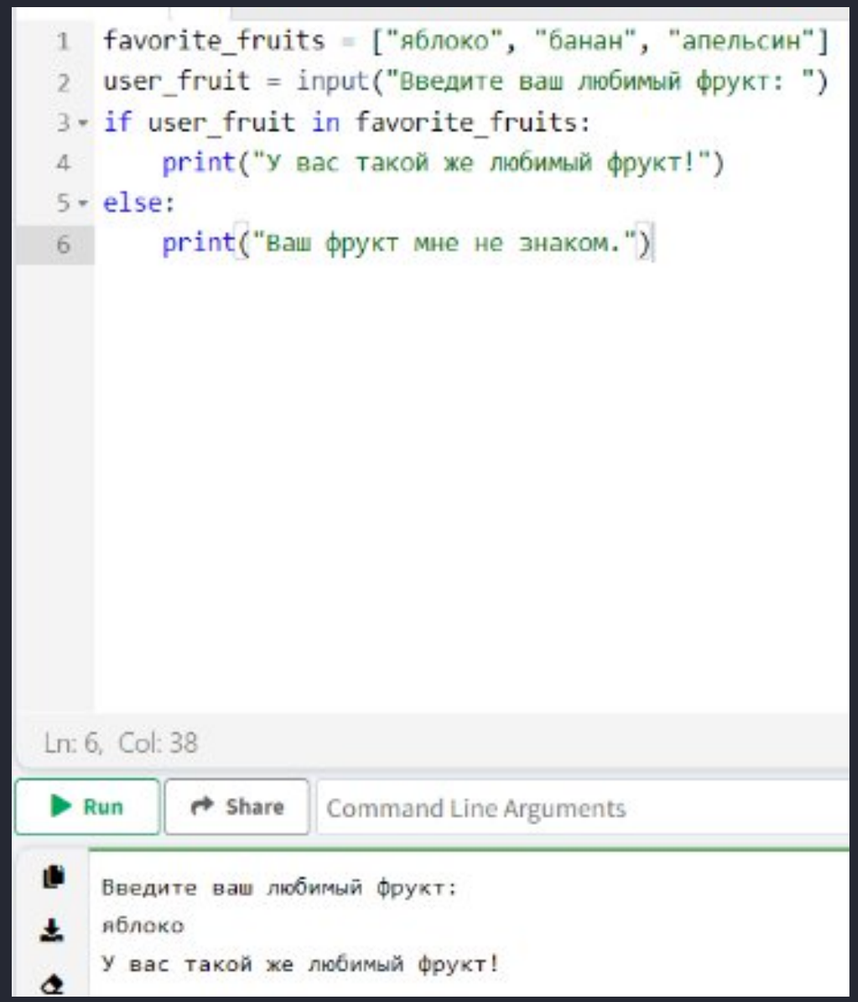
```
text = "Я изучаю Python и люблю программировать."
word = "Python"
if word in text:
    print("Слово найдено в тексте.")
else:
    print("Слово не найдено.")
```



Оператор вхождения **in** в Python используется для проверки, содержится ли элемент в последовательности (например, в списке, строке или кортеже). Если элемент присутствует, оператор возвращает **True**, в противном случае — **False**.

Оператор in

```python
favorite_fruits = ["яблоко", "банан", "апельсин"]
user_fruit = input("Введите ваш любимый фрукт: ")
if user_fruit in favorite_fruits:
    print("У вас такой же любимый фрукт!")
else:
    print("Ваш фрукт мне не знаком.")
```



Оператор вхождения **in** в Python используется для проверки, содержится ли элемент в последовательности (например, в **списке**, строке или кортеже).

Оператор in Функция len() Цикл for

```
names = ['Павел', 'Петр', 'Анна']
for name in names:
    print('Имя' ,name, 'имеет длину', len(name))


#Вывод
Имя Павел имеет длину 5
Имя Петр имеет длину 4
Имя Анна имеет длину 4
```

Оператор вхождения **in** в Python используется для проверки, содержится ли элемент в последовательности (например, в **списке**, строке или кортеже).

```
fruits = ['яблоко','апельсин','груша','ананас']

bakery_products = ['батон','булочка','хлеб','пироженое']

products = ['молоко','хлеб','яблоко','соль']


print('фрукты:')

for i in products:

    if i in fruits:

        print(i)


print('хлебобулочные изделия: ')

for i in products:

    if i in bakery_products:

        print(i)
```

▶ Содержимое файла `file.txt`

```
1  fruits = ['яблоко','апельсин','груша','ананас']
2
3  bakery_products = ['батон','булочка','хлеб','пироженое']
4
5  products = ['молоко','хлеб','яблоко','соль']
6
7  print('фрукты:')
8  for i in products:
9      if i in fruits:
10         print(i)
11
12 print('хлебобулочные изделия:')
13 for i in products:
14     if i in bakery_products:
15         print(i)
```

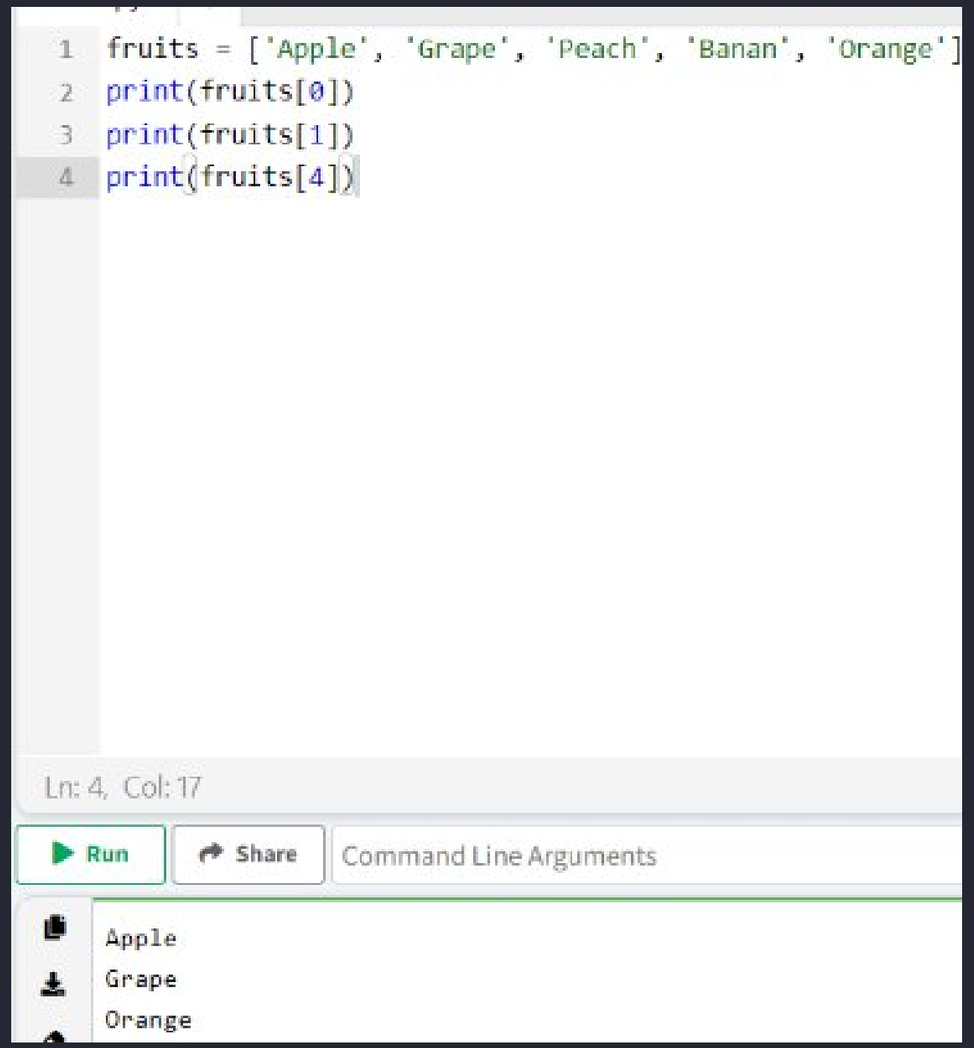
Выполнить!

```
фрукты:
яблоко
хлебобулочные изделия:
хлеб
```

Индексация

```
fruits = ['Apple', 'Grape', 'Peach', 'Banan', 'Orange']
print(fruits[0])
print(fruits[1])
print(fruits[4])

#Вывод
 Apple
Grape
Orange
```



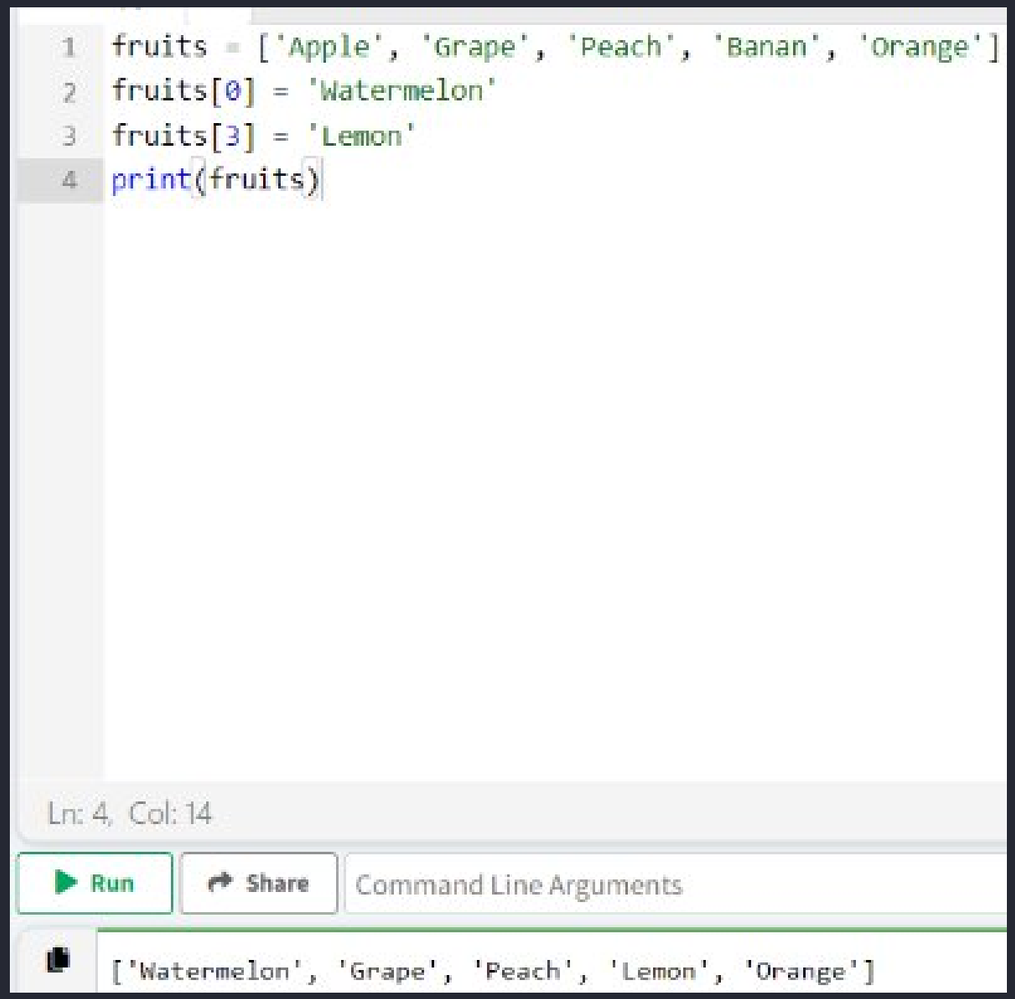
Индексация операция обращения к элементу по его порядковому номеру. Списки в Python индексируются, начиная с **0**. Вы можете обращаться к элементам списка по их индексам, чтобы получать, изменять или заменять их.

Индексация

#Списки в Python являются **изменяемым** типом данных

```
fruits = ['Apple', 'Grape', 'Peach', 'Banan', 'Orange']
fruits[0] = 'Watermelon'
fruits[3] = 'Lemon'
print(fruits)

#Вывод
['Watermelon', 'Grape', 'Peach', 'Lemon', 'Orange']
```



Индексация операция обращения к элементу по его порядковому номеру. Списки в Python индексируются, начиная с **0**. Вы можете обращаться к элементам списка по их индексам, чтобы получать, изменять или заменять их.

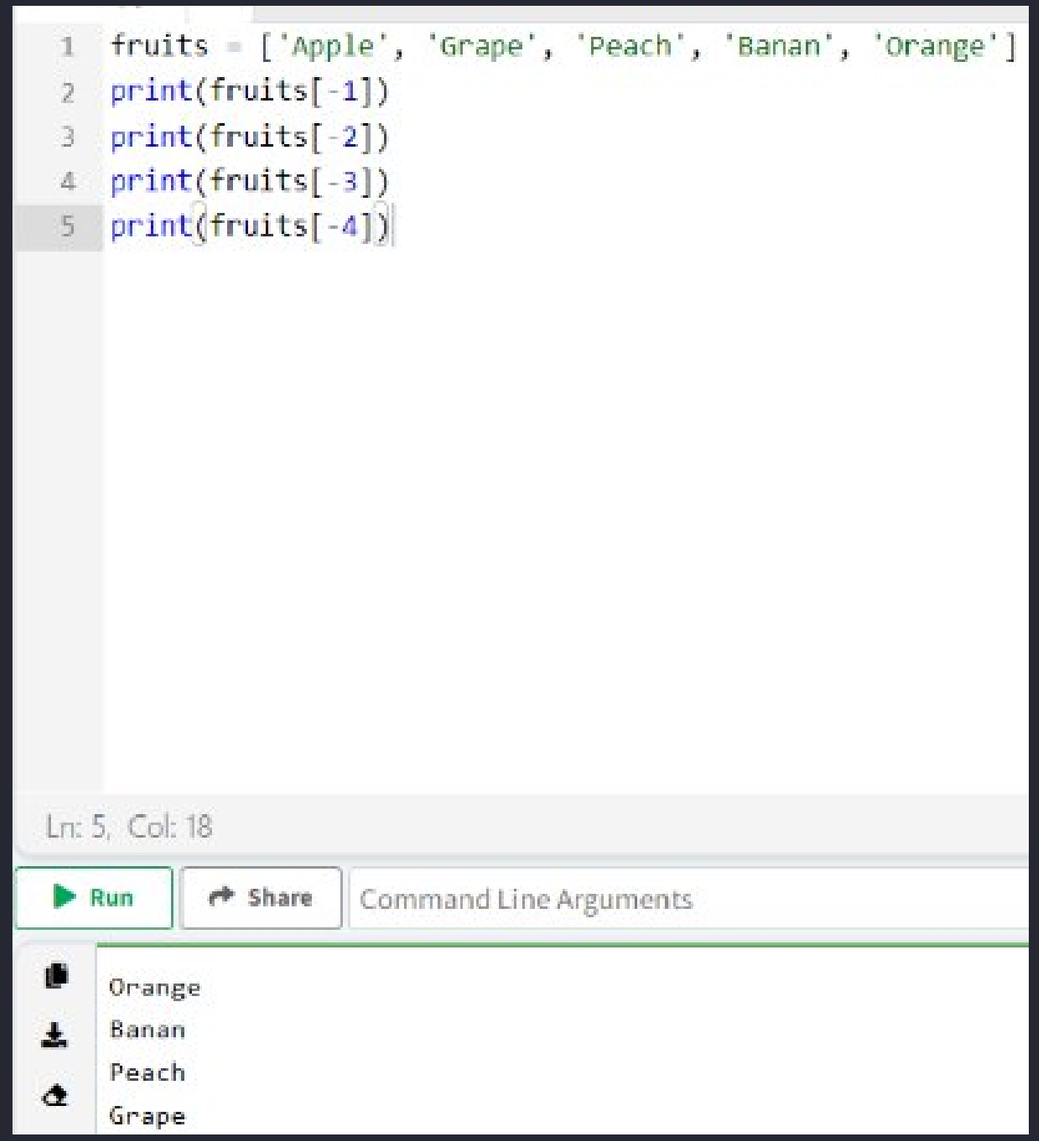
Индексация

#Индексирование работает и в обратную сторону.

```
fruits = ['Apple', 'Grape', 'Peach', 'Banan', 'Orange']
print(fruits[-1])
print(fruits[-2])
print(fruits[-3])
print(fruits[-4])
```

#Вывод

```
>>> Orange
>>> Banan
>>> Peach
>>> Grape
```



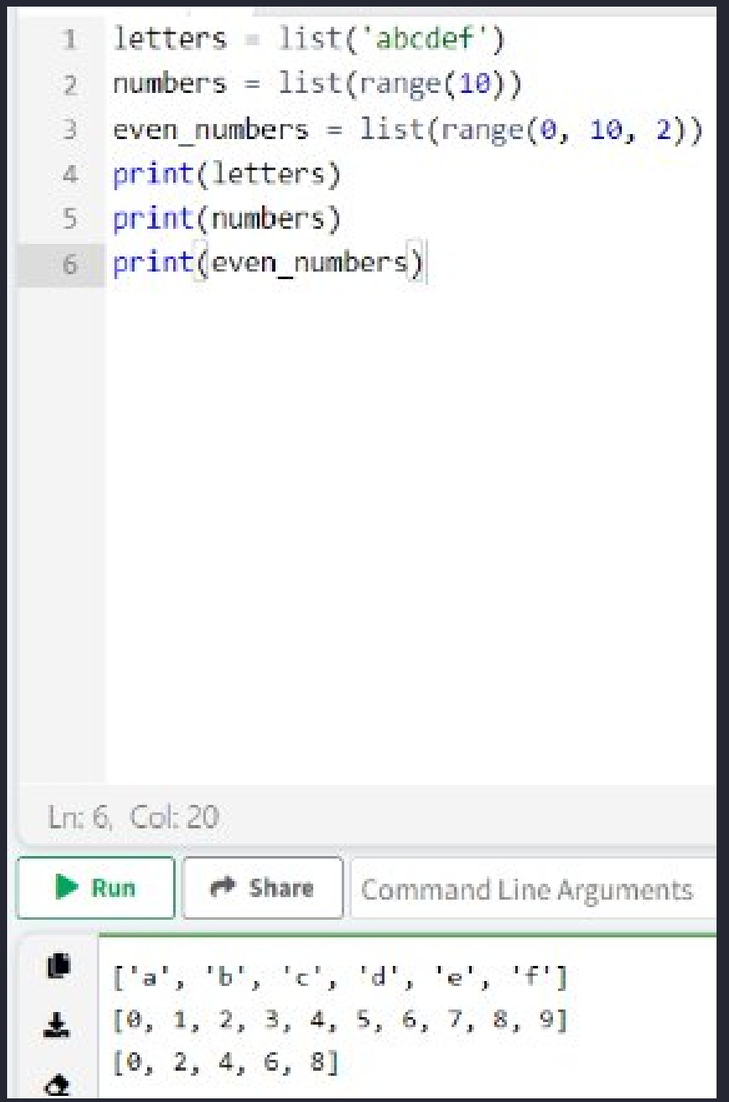
Индекс с номером **-1** дает нам доступ к **последнему** элементу, -2 к предпоследнему и т.д.

Создание списка с помощью list()

```
letters = list('abcdef')
numbers = list(range(10))
even_numbers = list(range(0, 10, 2))
print(letters)
print(numbers)
print(even_numbers)
```

#Вывод

```
>>> ['a', 'b', 'c', 'd', 'e', 'f'
>>> [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> [0, 2, 4, 6, 8]
```



Длина списка

```
fruits = ['Apple', 'Grape', 'Peach', 'Banan', 'Orange']
print(len(fruits))
```

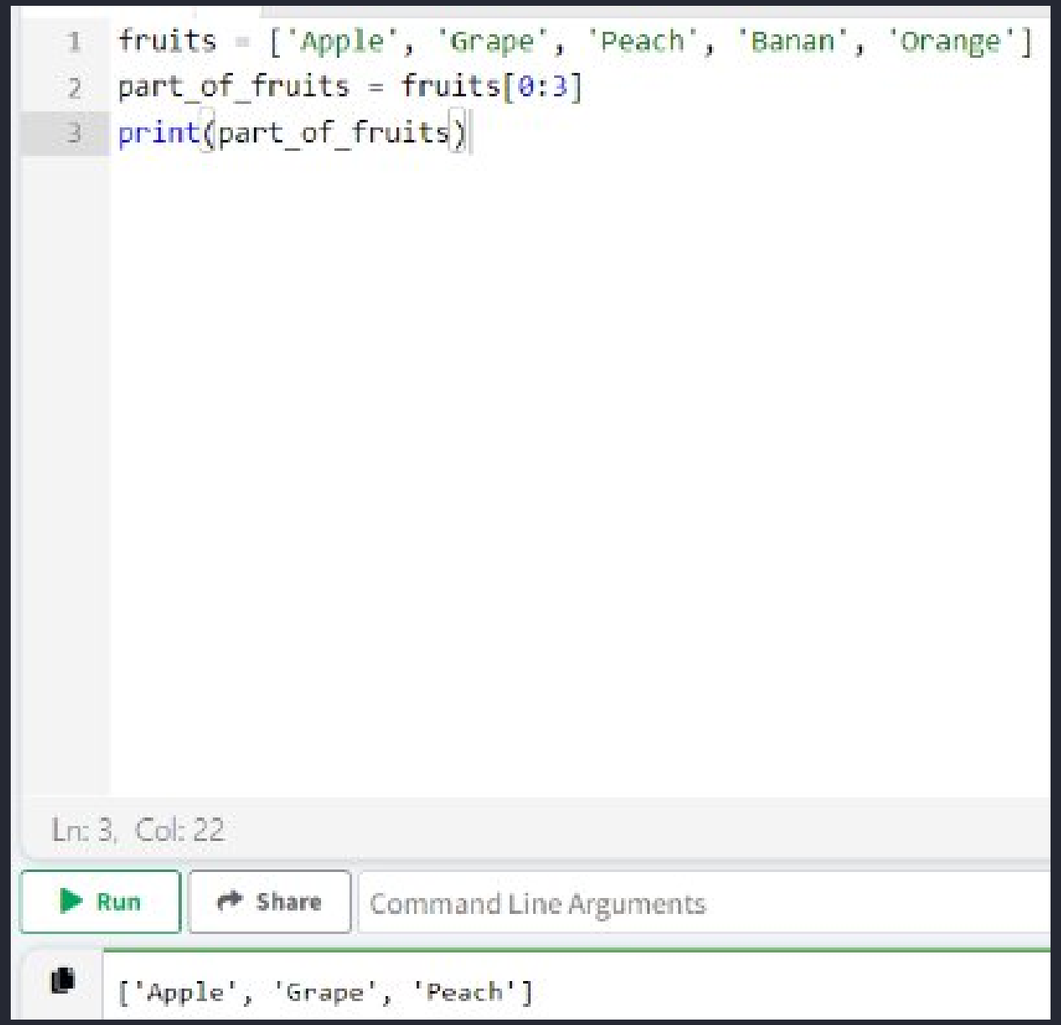
#Вывод

```
>>> 5
```

Срез списка

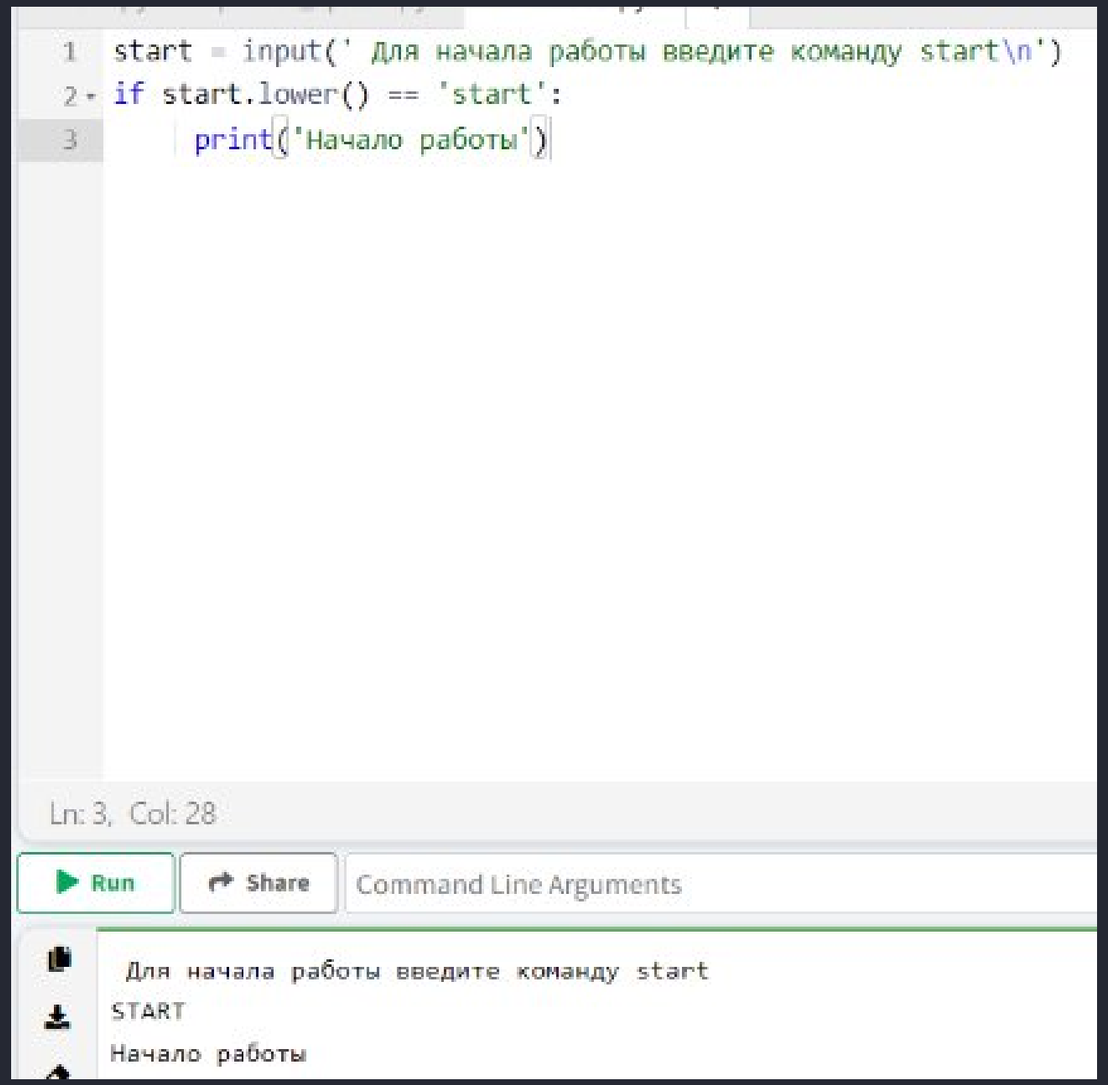
```
fruits = ['Apple', 'Grape', 'Peach', 'Banan', 'Orange']
part_of_fruits = fruits[0:3]
print(part_of_fruits)

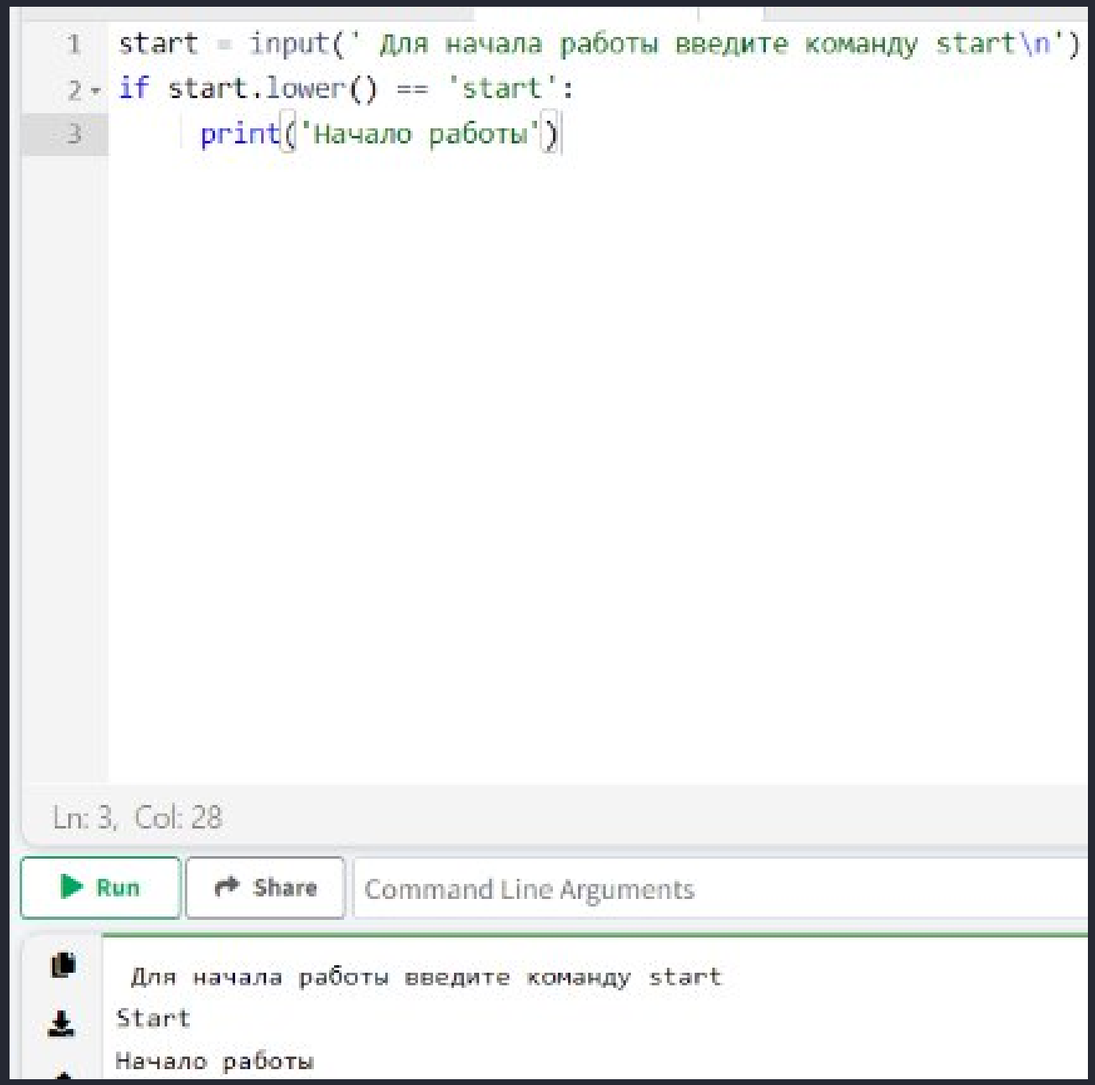
#Вывод
['Apple', 'Grape', 'Peach']
```



Срезом называется некоторая подпоследовательность. "Выбираем" часть от исходной последовательности элемента, не меняя её при этом. Срезы могут быть полезны при работе с большими массивами данных, когда необходимо быстро выбрать часть данных для обработки или анализа.

Метод lower()





Метод **lower()** - переводит все символы строки в нижний регистр, делая ее полностью строчной.

Метод lower()

```
1 product_list = []
2 product = input('Введите продукт: ')
3
4 product_list.append(product.lower())
5 print(product_list)
```

Ln: 5, Col: 20

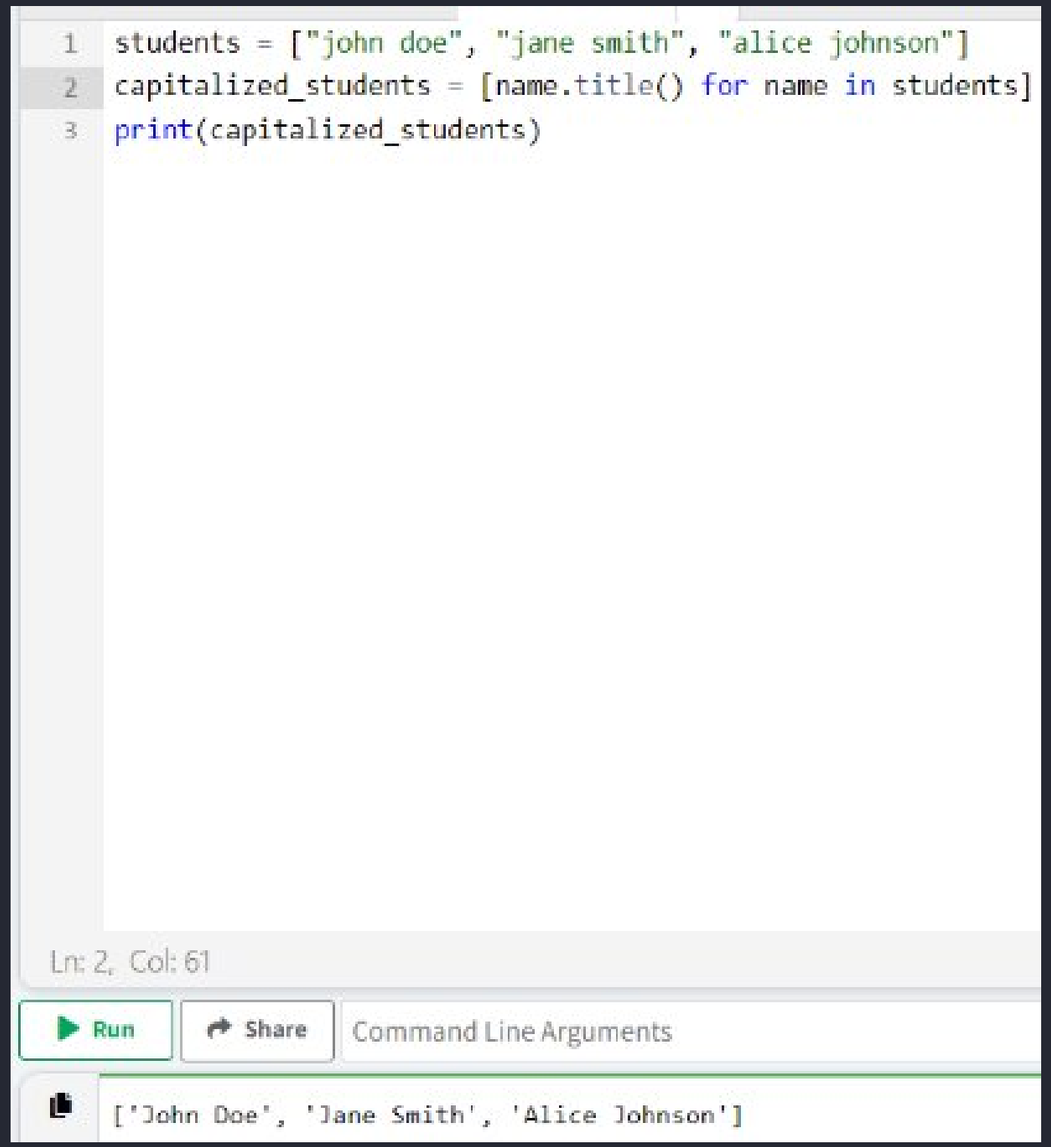
Run | Share | Command Line Arguments

```
Введите продукт:
ХЛЕБ
['хлеб']
```



Метод **lower()** - переводит **все символы** строки в нижний регистр, делая ее полностью строчной.

Метод **title()**




```
# Преобразование списка имен студентов,
# чтобы каждое имя начиналось с заглавной буквы.

students = ["john doe", "jane smith", "alice johnson"]
capitalized_students = [name.title() for name in students]
print(capitalized_students)
```

Метод **title()** преобразует **первый** символ каждого слова в **верхний регистр**.

Метод **title()**

```
1 surname_list = []
2 surname = input('Введите фамилию: ')
3
4 surname_list.append(surname.title())
5 print(surname_list)
```

Ln: 5, Col: 14

Run | Share | Command Line Arguments

```
Введите фамилию:
иванова
['Иванова']
```

```
1 surname_list = []
2 surname = input('Введите фамилию: ')
3
4 surname_list.append(surname.title())
5 print(surname_list)
```

Ln: 5, Col: 14

Run | Share | Command Line Arguments

```
Введите фамилию:
ИВАНОВА
['Иванова']
```

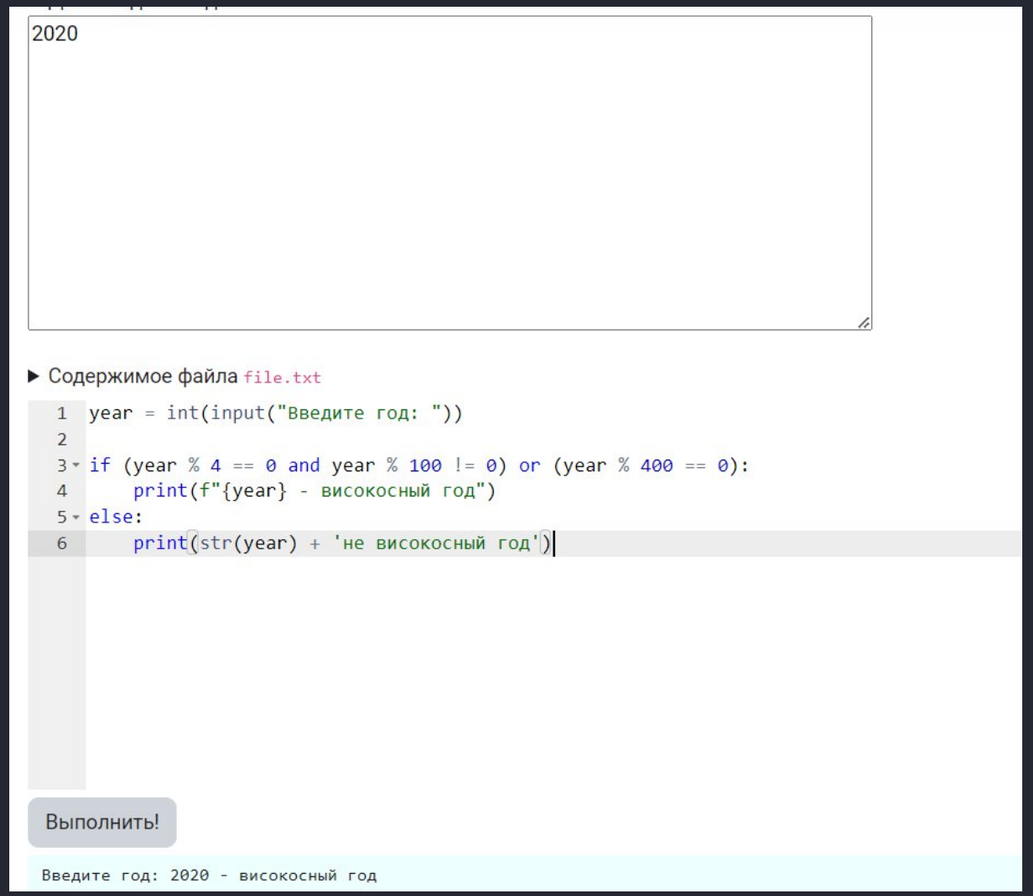
Метод **title()** преобразует **первый символ** каждого слова в **верхний регистр**, создавая так называемый "title case".

Метод **upper()**



Метод **upper()** преобразует все символы строки в **верхний регистр**, делая её полностью **заглавной**.

Форматирование строк с помощью f-strings



Позволяет в строку вставлять переменные.

print(f"{year} - високосный год")

Что такое f-strings?

F-strings, или "formatted string literals", - это улучшенный способ форматирования строк в Python, представленный в версии 3.6. Они позволяют легко включать значения переменных и выражений прямо внутри строк.

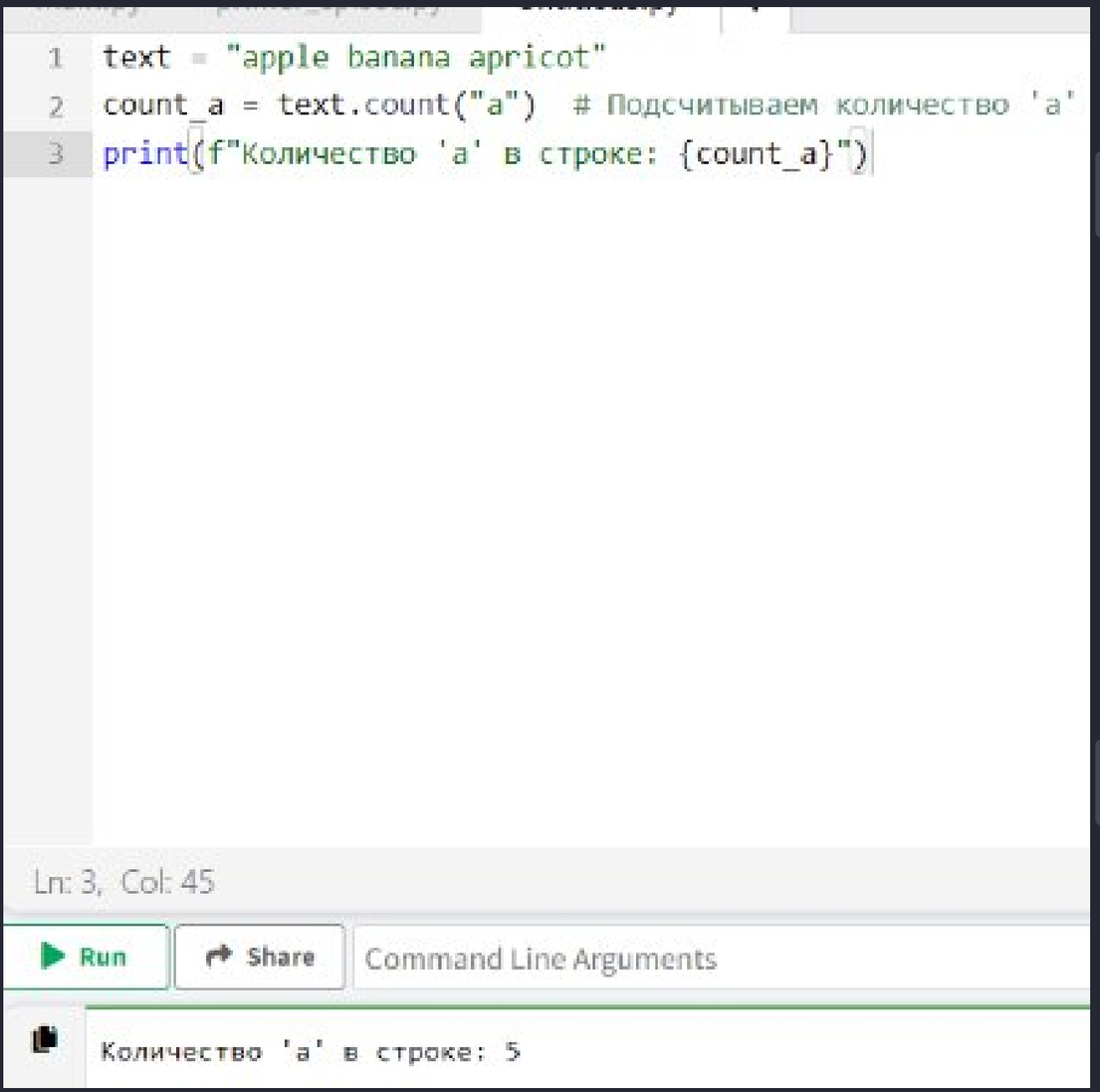
Синтаксис f-strings

Для использования f-strings, строку нужно начинать с буквы "**f**" или "**F**" перед открывающими кавычками. Значения переменных и выражений заключаются в **фигурные скобки** внутри строки.

Преимущества f-strings

f-strings делают код более читаемым и компактным по сравнению с традиционными методами форматирования, такими как .format() или % -операторы. Они также поддерживают дополнительные функции форматирования.

Метод count()



1 Подсчет символов

Метод **count()** возвращает количество вхождений заданной подстроки в строке. Это полезно для проверки частоты появления определенных символов или слов.

2 Регистрозависимость

По умолчанию **count()** учитывает регистр. Можно передать дополнительный аргумент, чтобы сделать поиск регистронезависимым.

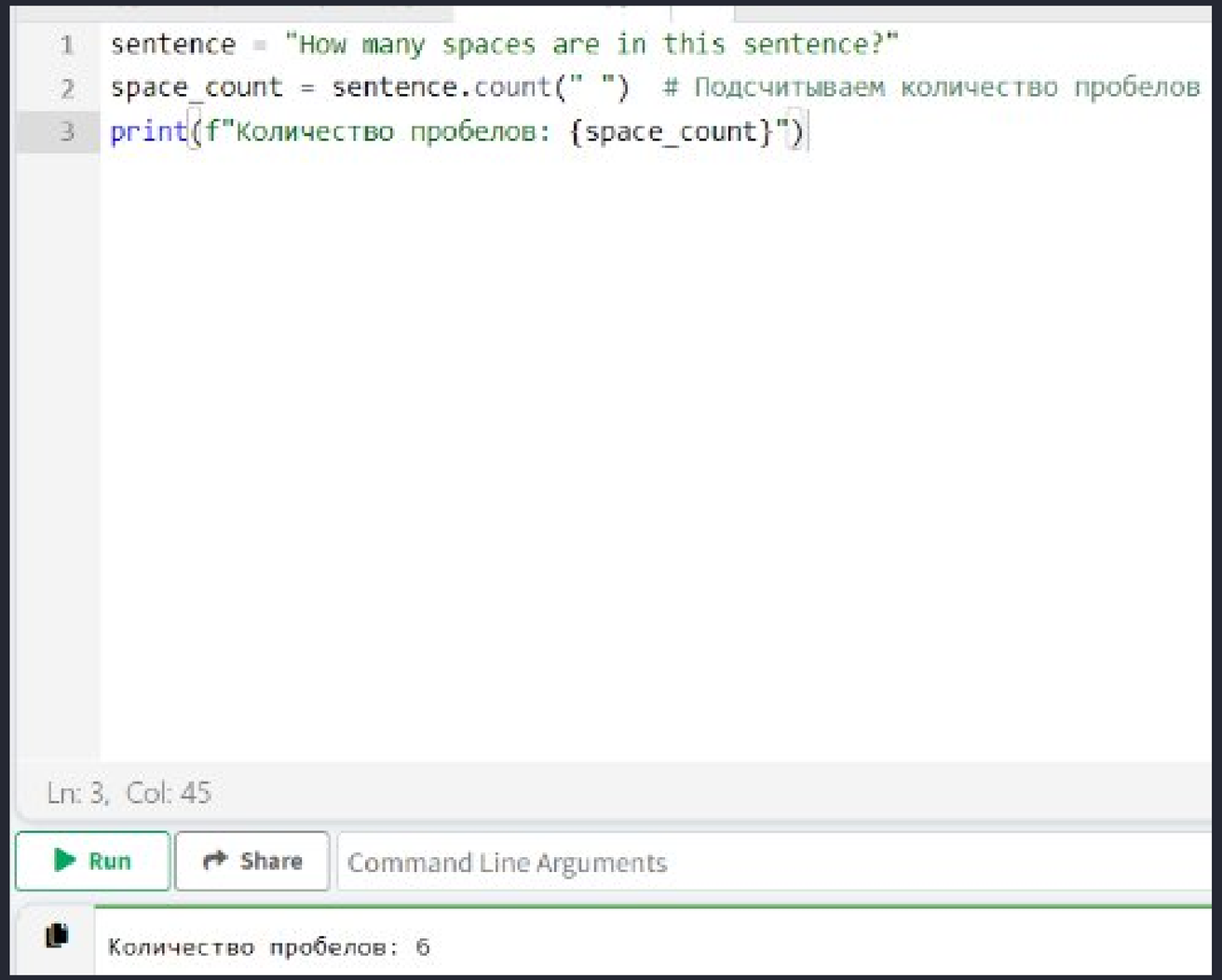
3 Счетчик вхождений

Результат **count()** - это целое число, показывающее, сколько раз найдена искомая подстрока. Это позволяет анализировать частоту встречаемости элементов в тексте.

Метод count()

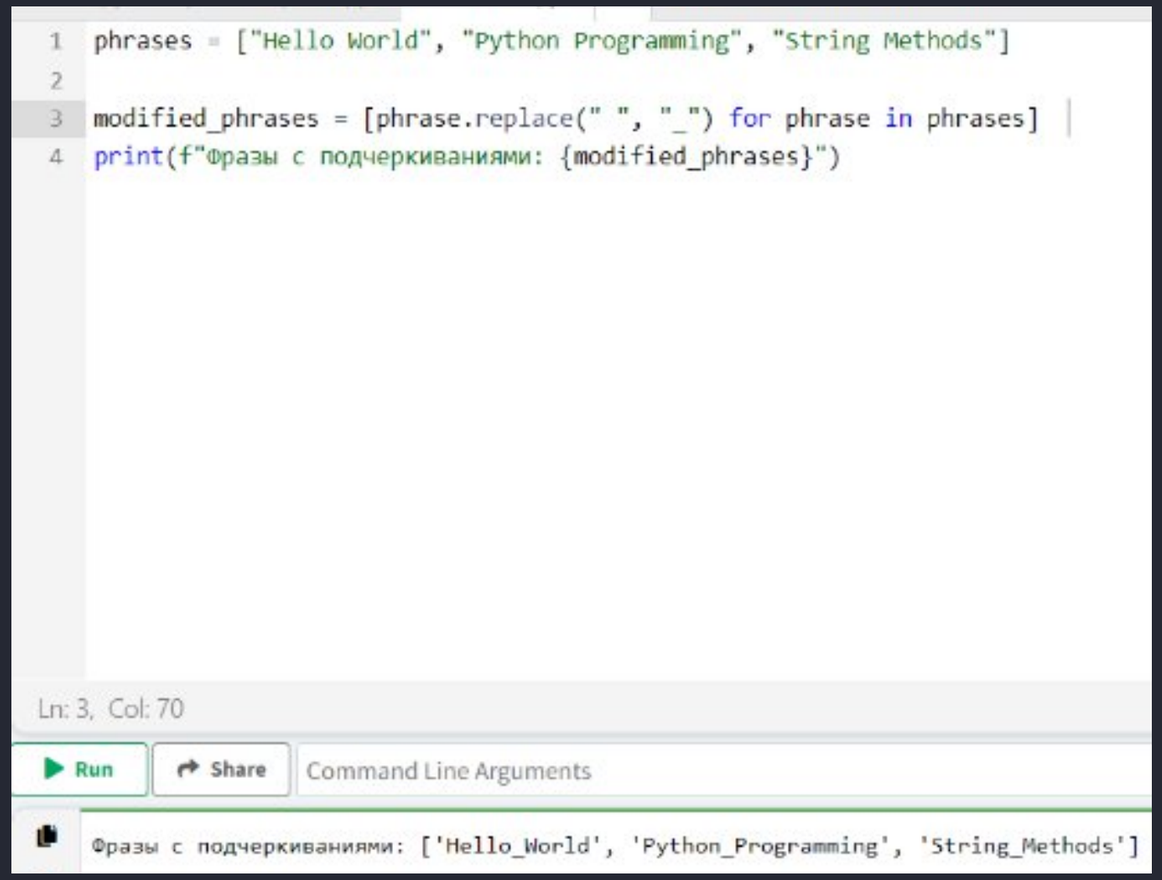
```
sentence = "How many spaces are in this sentence?"
space_count = sentence.count(" ")
print(f"Количество пробелов: {space_count}")


# Вывод
Количество пробелов: 6
```



Метод **count()** возвращает количество вхождений заданной подстроки в строке. Это полезно для проверки частоты появления определенных символов или слов.

Метод replace()



1

Замена подстрок

Метод **replace()** позволяет заменять одну подстроку на другую в исходной строке. Это очень полезно для выполнения различных преобразований текста, таких как исправление опечаток или унификация формата данных.

2

Регулируемая замена

replace() принимает два аргумента: подстроку для поиска и подстроку для замены. Можно также указать необязательный третий аргумент, чтобы ограничить количество замен.

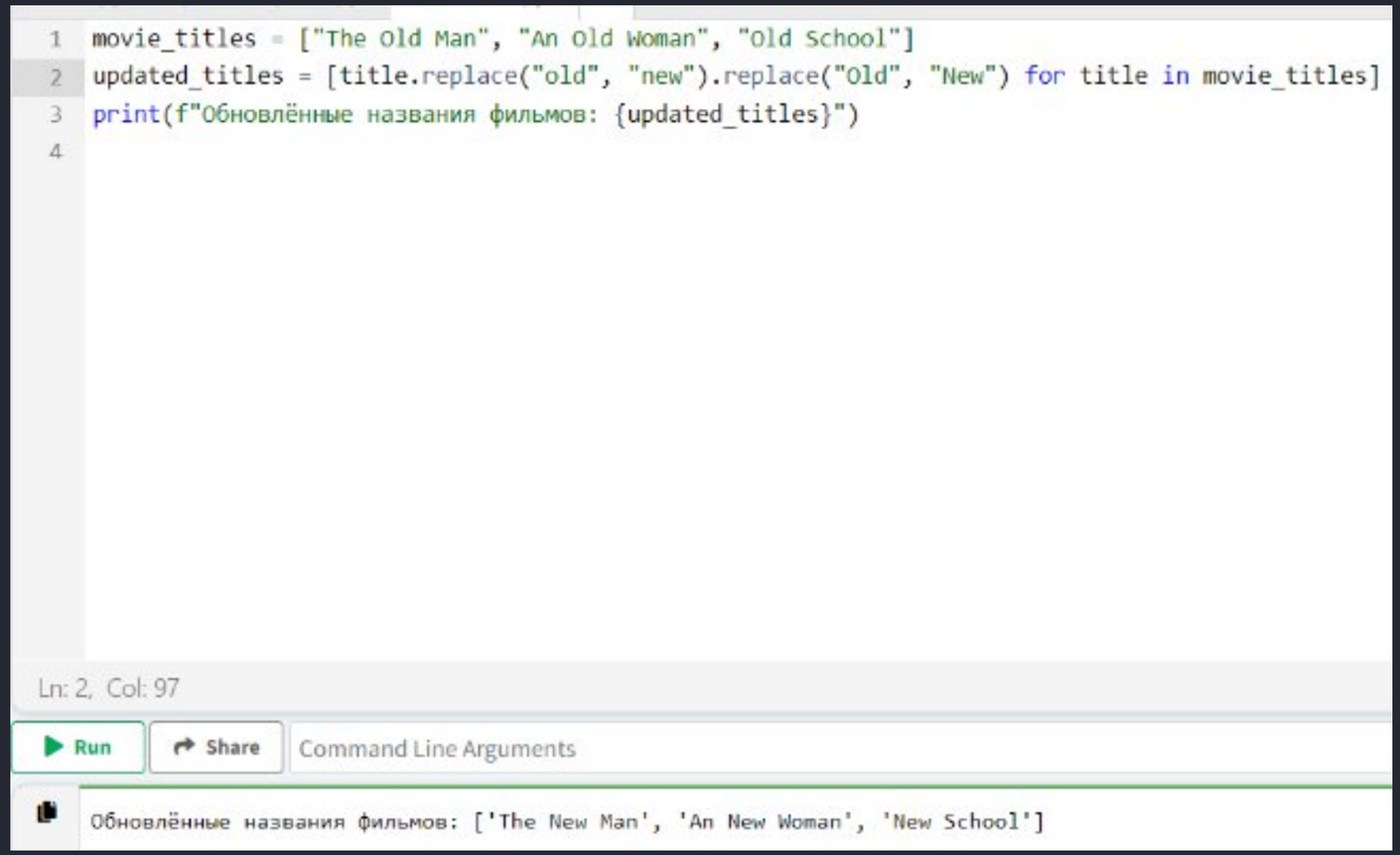
3

Применение к большим текстам

Метод **replace()** эффективен даже для работы с очень длинными строками, позволяя выполнять массовые замены быстро и без ошибок.

```
# Заменяем пробелы на подчеркивания
phrases = ["Hello World", "Python Programming", "String Methods"]
modified_phrases = [phrase.replace(" ", "_") for phrase in phrases]
print(f"Фразы с подчеркиваниями: {modified_phrases}")
#Вывод
Фразы с подчеркиваниями: ['Hello_World', 'Python_Programming', 'String_Methods']
```

Метод replace()



Заменяем "old" на "**new**"

movie_titles = ["The Old Man", "An Old Woman", "Old School"]

updated_titles = [title.**replace**("old", "new").replace("Old", "New") for title in movie_titles]

print(f"Обновлённые названия фильмов: {updated_titles}")

#Вывод

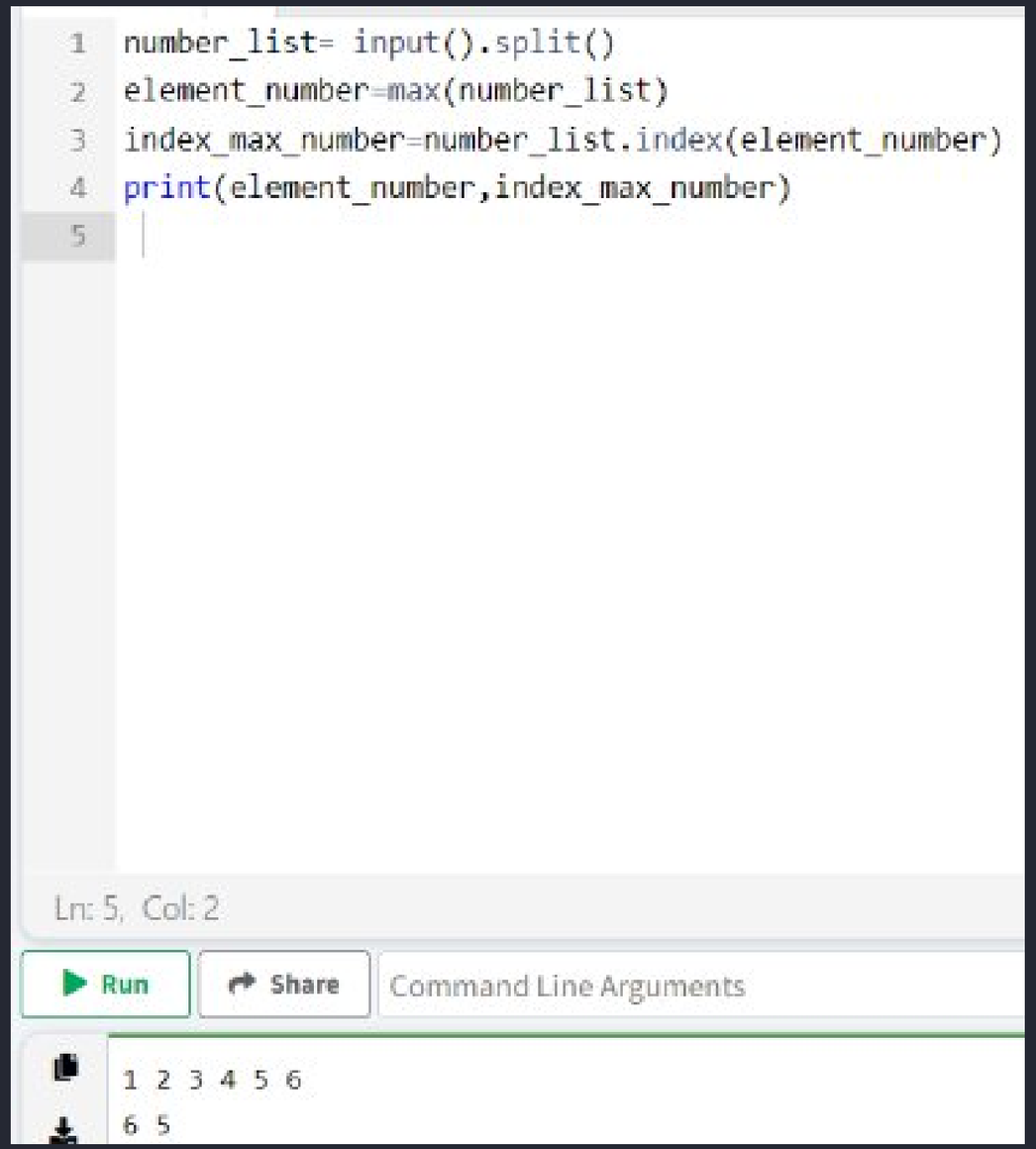
Обновлённые названия фильмов: ["The New Man", "An New Woman", " New School"]

Метод **replace()** позволяет заменять одну подстроку на другую в исходной строке.

Метод split()

```
# Найти наибольший элемент в списке и индекс этого элемента  в списке.
number_list = input('Введите ряд чисел').split()
element_number = max(number_list)
index_max_number = number_list.index(element_number)
print(element_number, index_max_number)


 #Вывод
1 2 3 4 5 6
6 5
```



Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

Метод split()

```
# Количество положительных элемента  в списке.

numbers_str = input('Введите ряд чисел: ')


numbers = list(map(int, numbers_str.split()))


count_positive_numbers = sum(1 for num in numbers if num > 0)

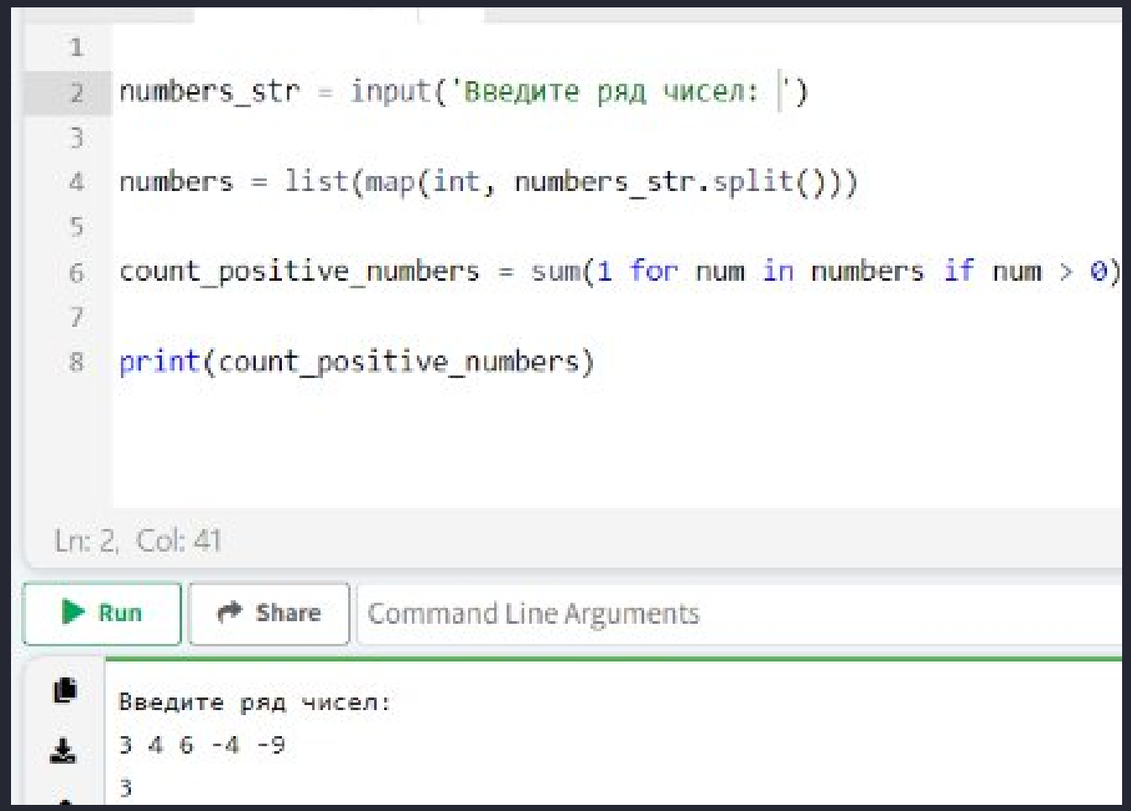

print(count_positive_numbers)


 #Вывод

Введите ряд чисел:

3 4 6 -4 -9

3
```



Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

```
# Количество положительных элемента  в списке.
numbers_str = input('Введите ряд чисел: ')

numbers = list(map(int, numbers_str.split()))

count_positive_numbers = sum(1 for num in numbers if num > 0)

print(count_positive_numbers)
```

Цифра 1 имеет важное значение. Это не просто число, а часть так называемого **генератора списков** (list comprehension).

Генератор списков позволяет создавать новый список на основе существующего, выполняя определённые условия и операции над каждым элементом исходного списка.

В данном случае генератор списков создаёт новый список, где каждый элемент равен 1, если соответствующий элемент исходного списка больше нуля (num > 0).

Этот генератор выглядит так: [1 for num in numbers if num > 0].
Затем функция sum() применяет операцию сложения ко всем элементам этого нового списка и возвращает итоговую сумму.
То есть sum([1, 1, 1]) равно 3, а sum([1, 1]) равно 2.

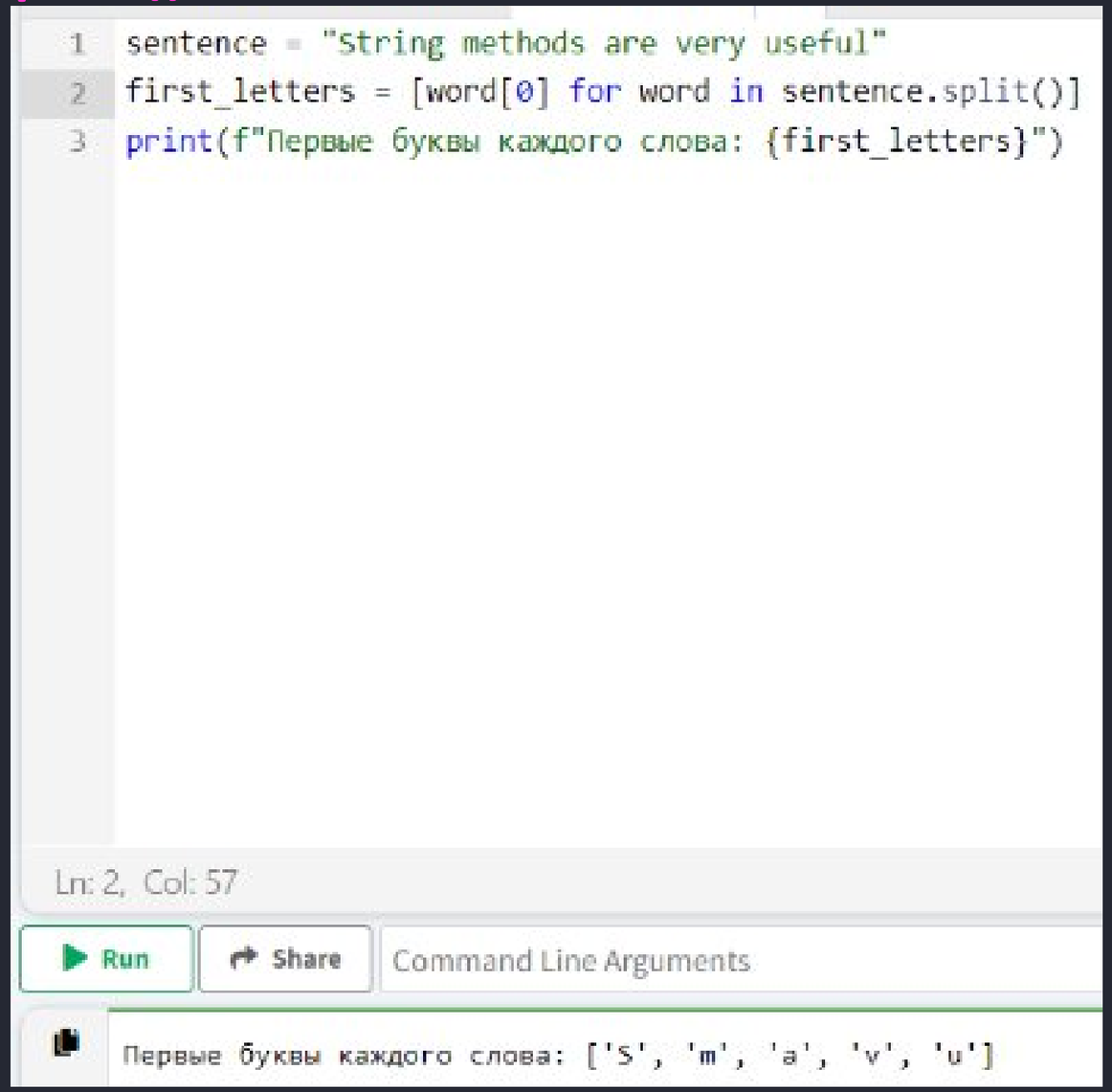
Таким образом, использование 1 внутри генератора списков необходимо для создания списка, содержащего количество положительных элементов исходного списка.

Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

Метод split()

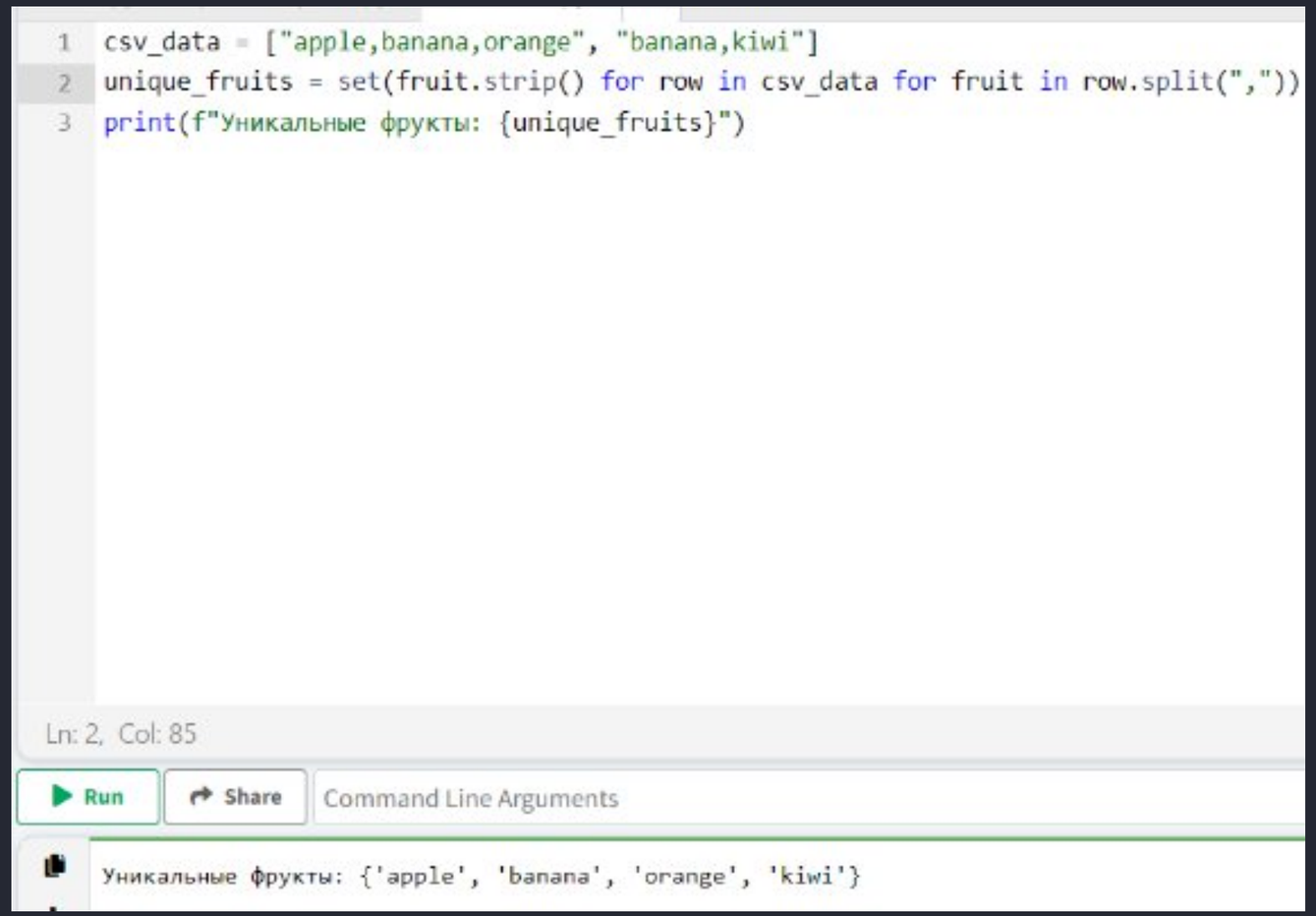
```
# Получаем первые буквы каждого слова


sentence = "String methods are very useful"
first_letters = [word[0] for word in sentence.split()]
print(f"Первые буквы каждого слова: {first_letters}")
#Вывод
Первые буквы каждого слова: ['S', 'm', 'a', 'v', 'u']
```



Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

Метод split()




```
# Разделяем по запятой

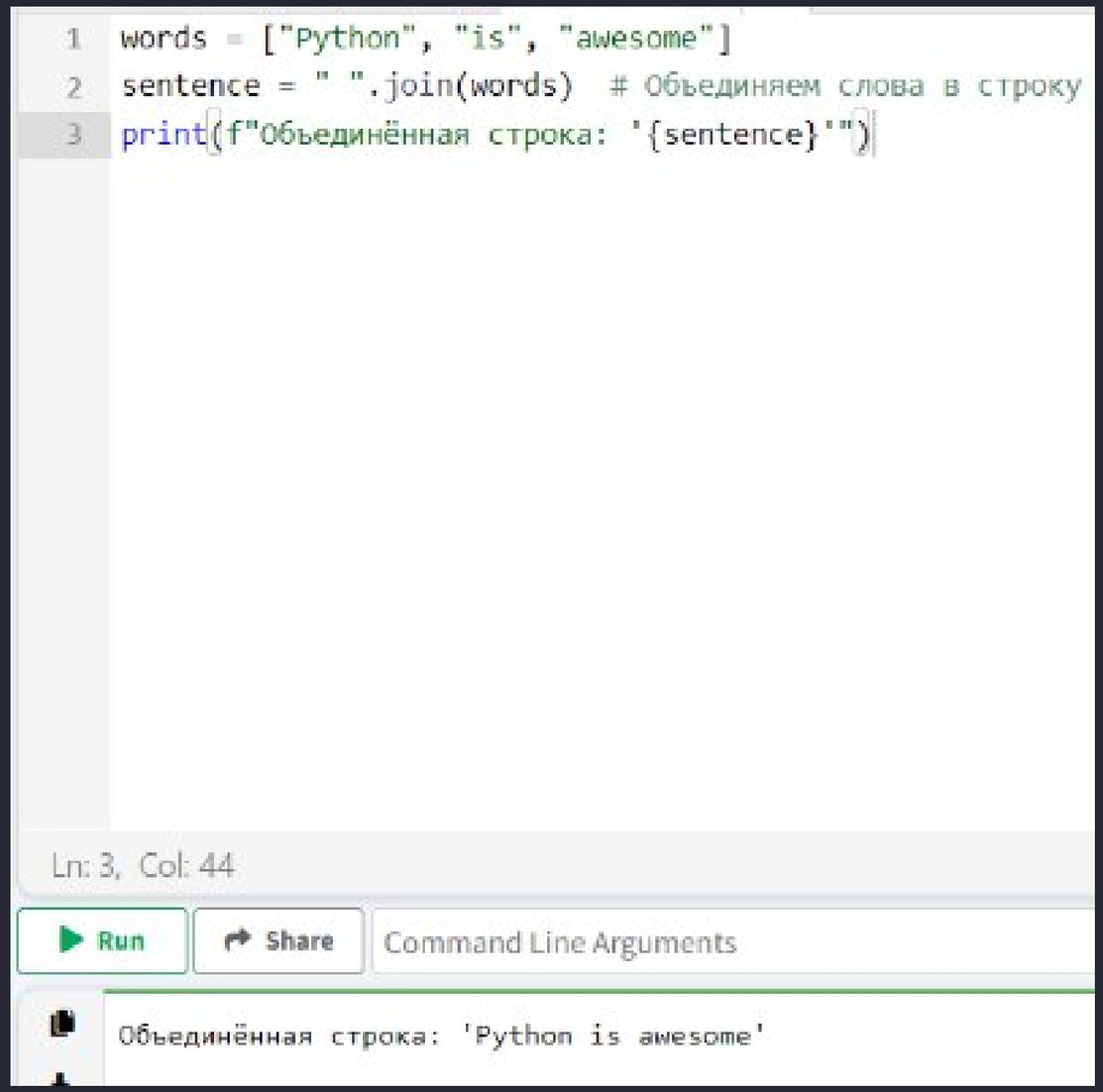

csv_data = ["apple,banana,orange", "banana,kiwi"]
unique_fruits = set(fruit.strip() for row in csv_data for fruit in row.split(","))
print(f"Уникальные фрукты: {unique_fruits}")
```

Метод **split()** используется для разделения строки на части (списки строк) по заданному разделителю.

Метод join()

```
# Объедините список слов в одну строку, разделяя их пробелами.
words = ["Python", "is", "awesome"]
sentence = " ".join(words)  # Объединяем слова в строку
print(f"Объединённая строка: '{sentence}'")

#Вывод
Объединённая строка: 'Python is awesome '
```

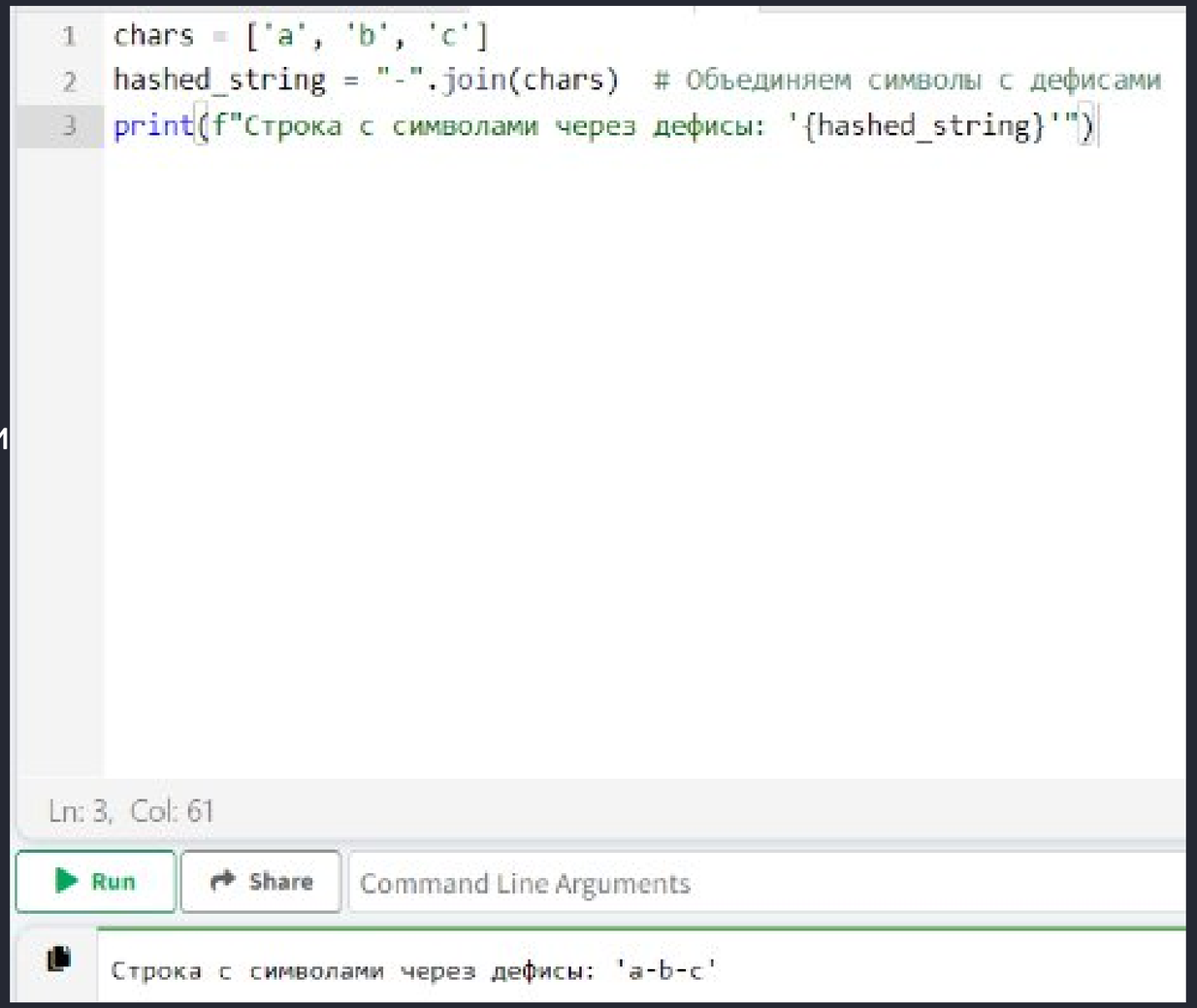


Метод **join()** используется для объединения списка строк в одну строку, используя указанный разделитель. Это удобно для создания строк из набора отдельных компонентов.

Метод join()

```
chars = ['a', 'b', 'c']
hashed_string = "-".join(chars)  # Объединяем символы с дефисами
print(f"Строка с символами через дефисы: '{hashed_string}'")


#Вывод
Строка с символами через дефисы: 'a-b-c'
```

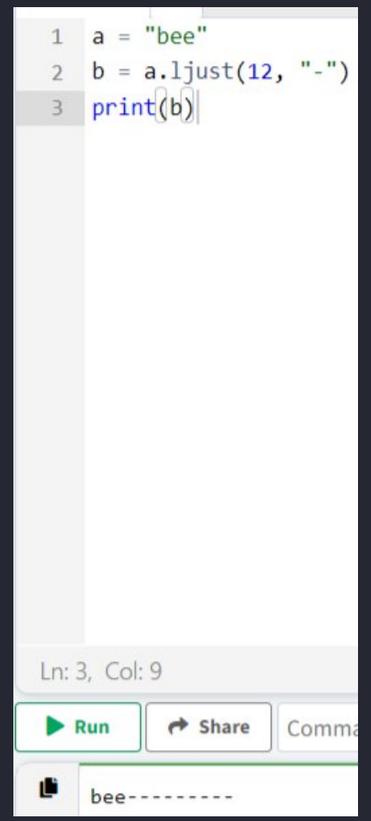


Метод **join()** используется для объединения списка строк в одну строку, используя указанный разделитель.

Метод ljust()

```
a = "bee"
b = a.ljust(12, "-")
print(b)

#Вывод
bee---------
```



Метод **ljust(**width[, fillchar])– возвращает строку, выровненную по левому краю в рамках большей строки (ее ширина в символах задается при помощи параметра width).

Для заполнения места, не занятого оригинальной строкой, используются ASCII пробелы, но можно и указать желаемый символ — при помощи параметра **fillchar**.

Если заданная ширина строки меньше или равна длине оригинальной строки (определяется при помощи **len(s)**), то возвращается оригинальная строка.

Здесь все по аналогии с методом **center()**, только там строка выравнивается по центру.

Метод rjust()

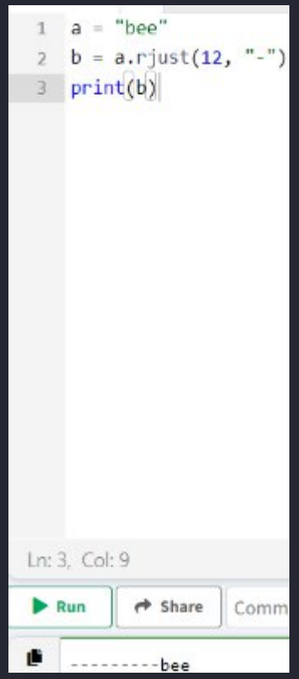
```
a = "bee"

b = a.rjust(12, "-")

print(b)


#Вывод

---------bee
```



Метод **rjust(**width[, fillchar]**)** – возвращает строку, выровненную по правому краю в рамках большей строки (ее ширина в символах задается при помощи параметра width).

Для заполнения места, не занятого оригинальной строкой, используются символы, заданные при помощи параметра fillchar.

Если символ-заполнитель не задан, по умолчанию будут использоваться ASCII пробелы.

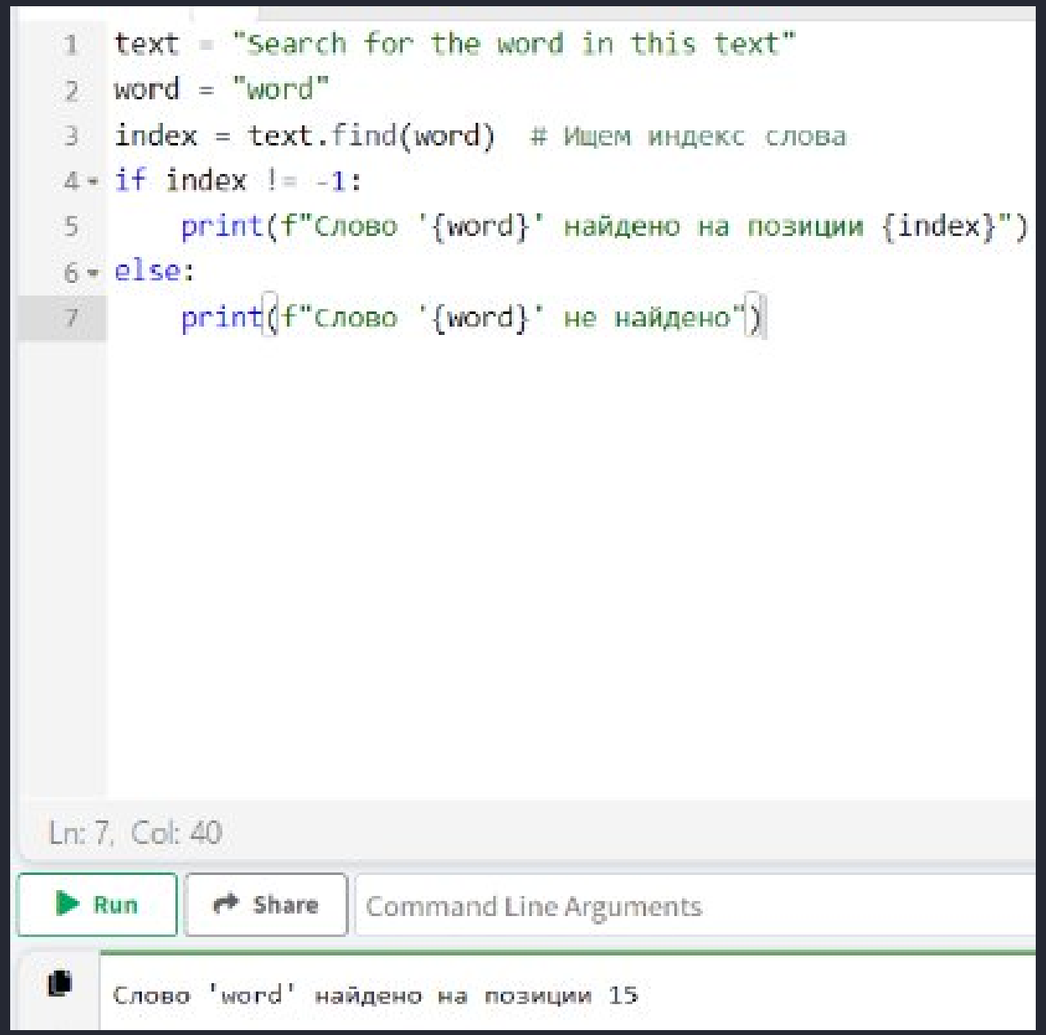
В случае, если заданная ширина строки меньше или равна длине оригинальной строки (определяется при помощи len(s)), возвращается оригинальная строка.

Здесь все по аналогии с методом center(), где строка выравнивается по центру, и с методом ljust(), где строка выравнивается по левому краю.

Метод find()

```
text = "Search for the word in this text"
word = "word"
index = text.find(word)  # Ищем индекс слова
if index != -1:
    print(f"Слово '{word}' найдено на позиции {index}")
else:
    print(f"Слово '{word}' не найдено")

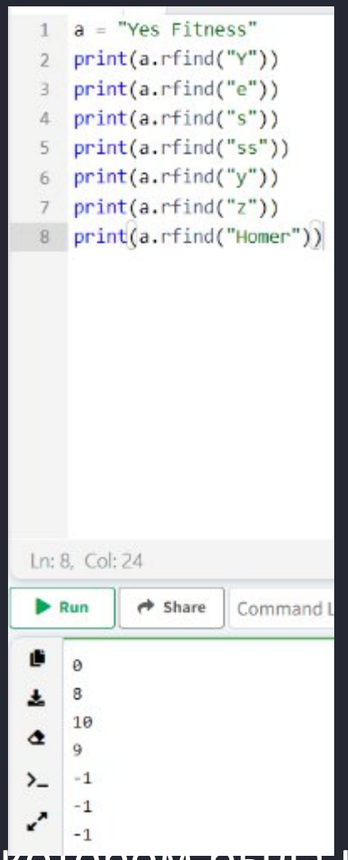

# Вывод
Слово 'word' найдено на позиции 15
```



Метод **find()** находит индекс первого вхождения подстроки в строке. Возвращает -1, если подстрока не найдена.

Метод **rfind()**

```
a = "Yes Fitness"
print(a.rfind("Y"))
print(a.rfind("e"))
print(a.rfind("s"))
print(a.rfind("ss"))
print(a.rfind("y"))
print(a.rfind("z"))
print(a.rfind("Homer"))
```



Метод **rfind(**sub[, start[, end]]**)** возвращает **самый большой индекс** в строке, на котором была найдена указанная подстрока. При помощи опциональных аргументов **start** и **end** можно задать диапазон для поиска подстроки в строке (эти аргументы интерпретируются как срез). Если подстрока в строке **не найдена**, возвращается **-1.**

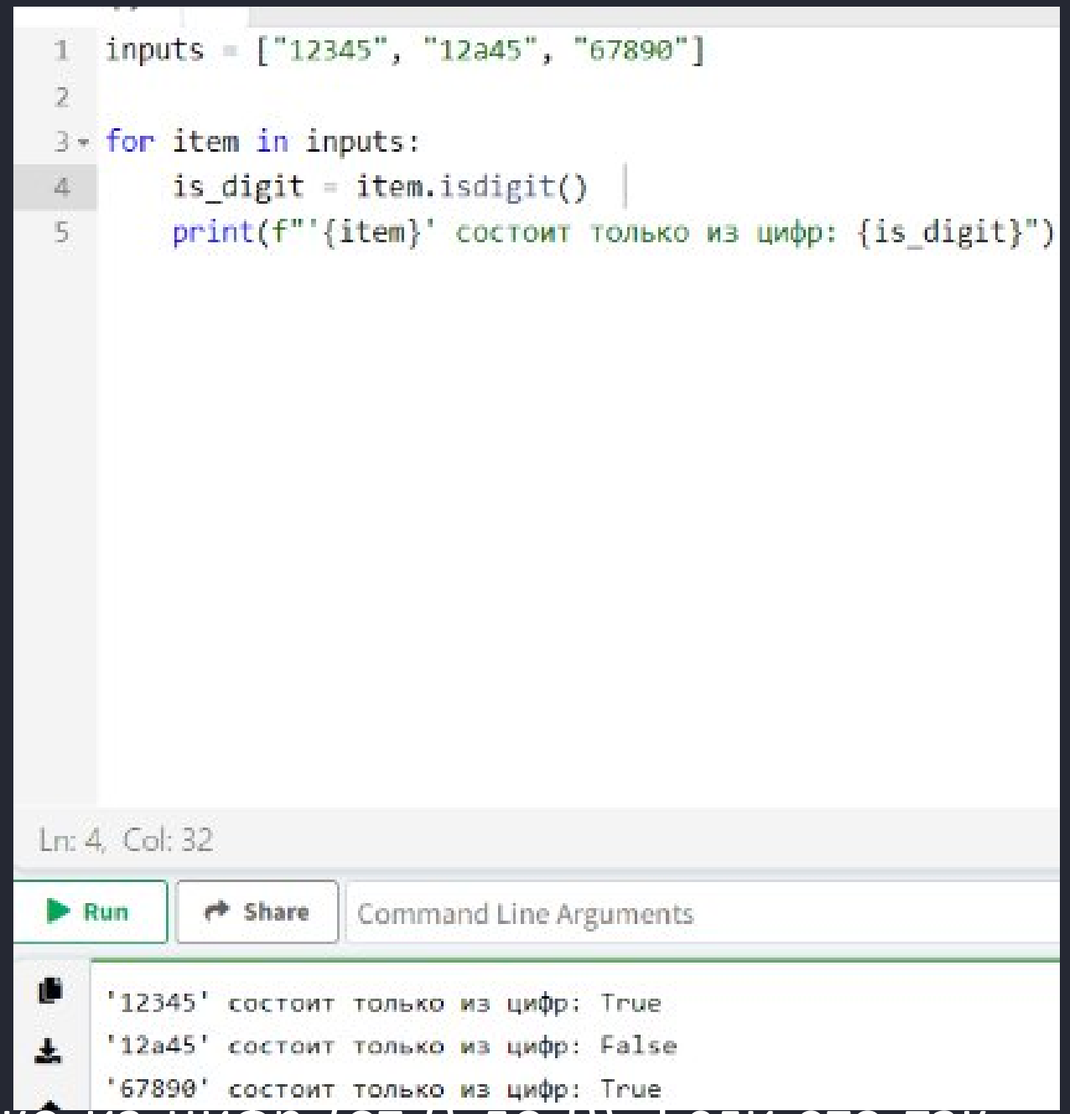
Работа этого метода аналогична работе метода **find(),** только find() ищет индекс *первого* вхождения подстроки.

Метод **isdigit()**

```
# Проверяем, состоит ли строка только из цифр
inputs = ["12345", "12a45", "67890"]


for item in inputs:
    is_digit = item.isdigit()
    print(f"'{item}' состоит только из цифр: {is_digit}")


#Вывод
'12345' состоит только из цифр:
True '12a45' состоит только из цифр:
False '67890' состоит только из цифр: True
```

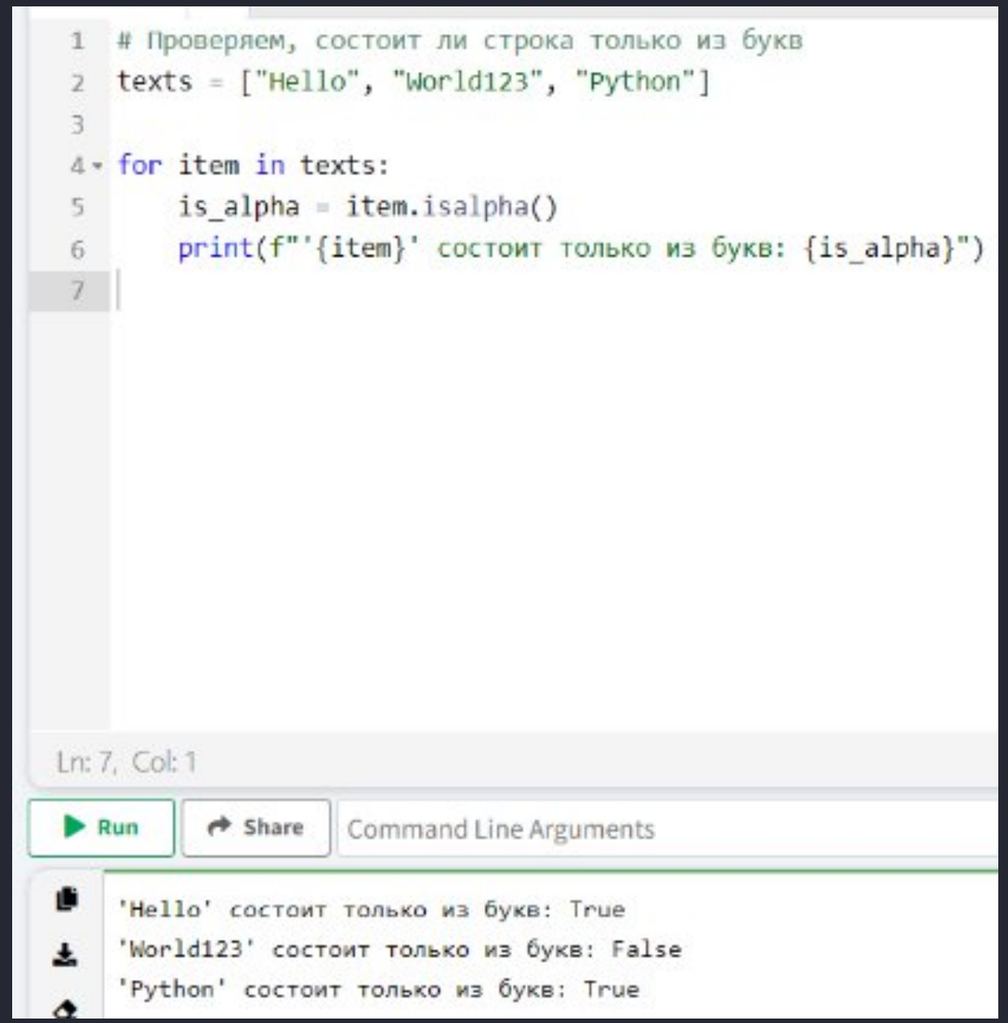


Метод **isdigit()** проверяет, состоит ли строка только из цифр (от 0 до 9). Если это так, функция возвращает True, иначе False.

Метод **isalpha()**

```
# Проверяем, состоит ли строка только из букв
texts = ["Hello", "World123", "Python"]


for item in texts:
    is_alpha = item.isalpha()
    print(f"'{item}' состоит только из букв: {is_alpha}")
```

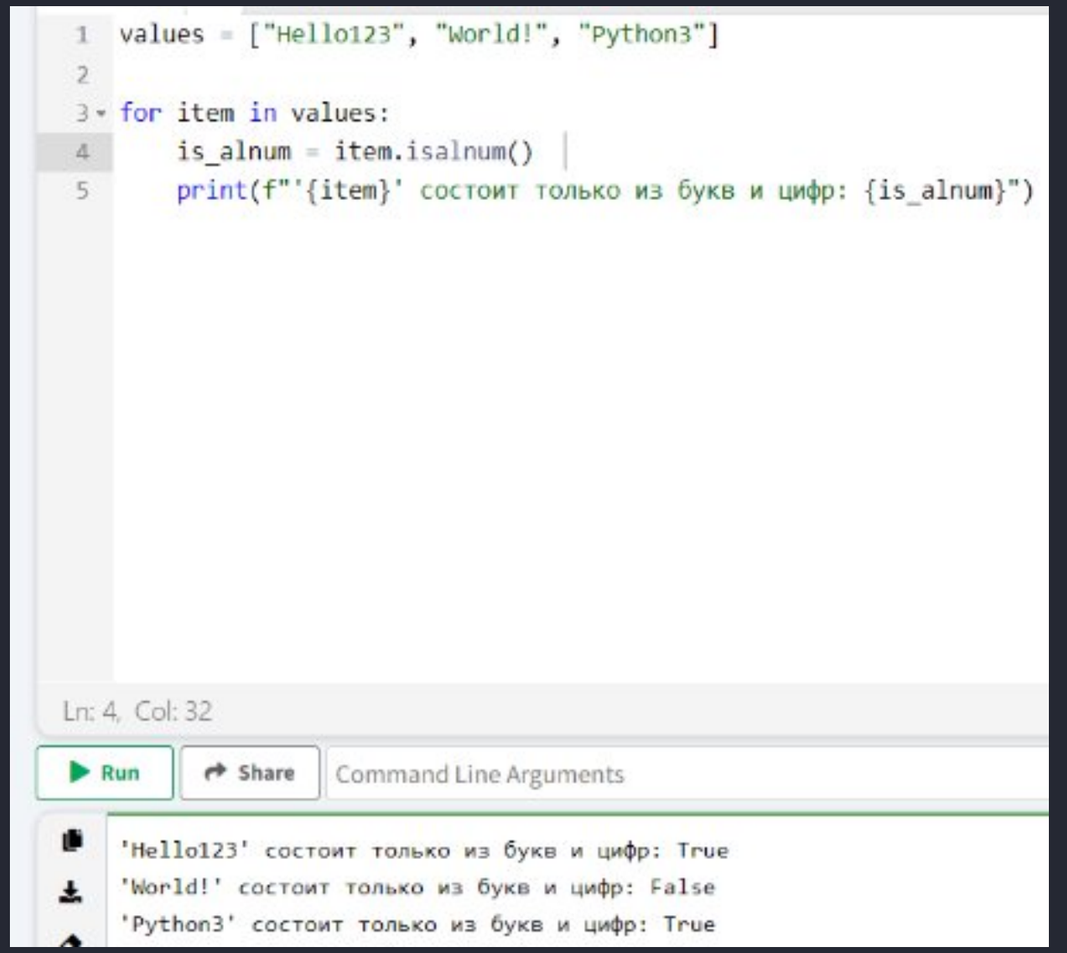


Метод **isalpha()** проверяет, состоит ли строка только из алфавитных символов (букв латинского алфавита, больших и маленьких) без пробелов, знаков препинания и других специальных символов. Если строка состоит только из букв, функция возвращает True, иначе False.

Метод isalnum()

```
# Проверяем, состоит ли строка из букв и цифр
values = ["Hello123", "World!", "Python3"]

for item in values:
    is_alnum = item.isalnum()
    print(f"'{item}' состоит только из букв и цифр: {is_alnum}")
```

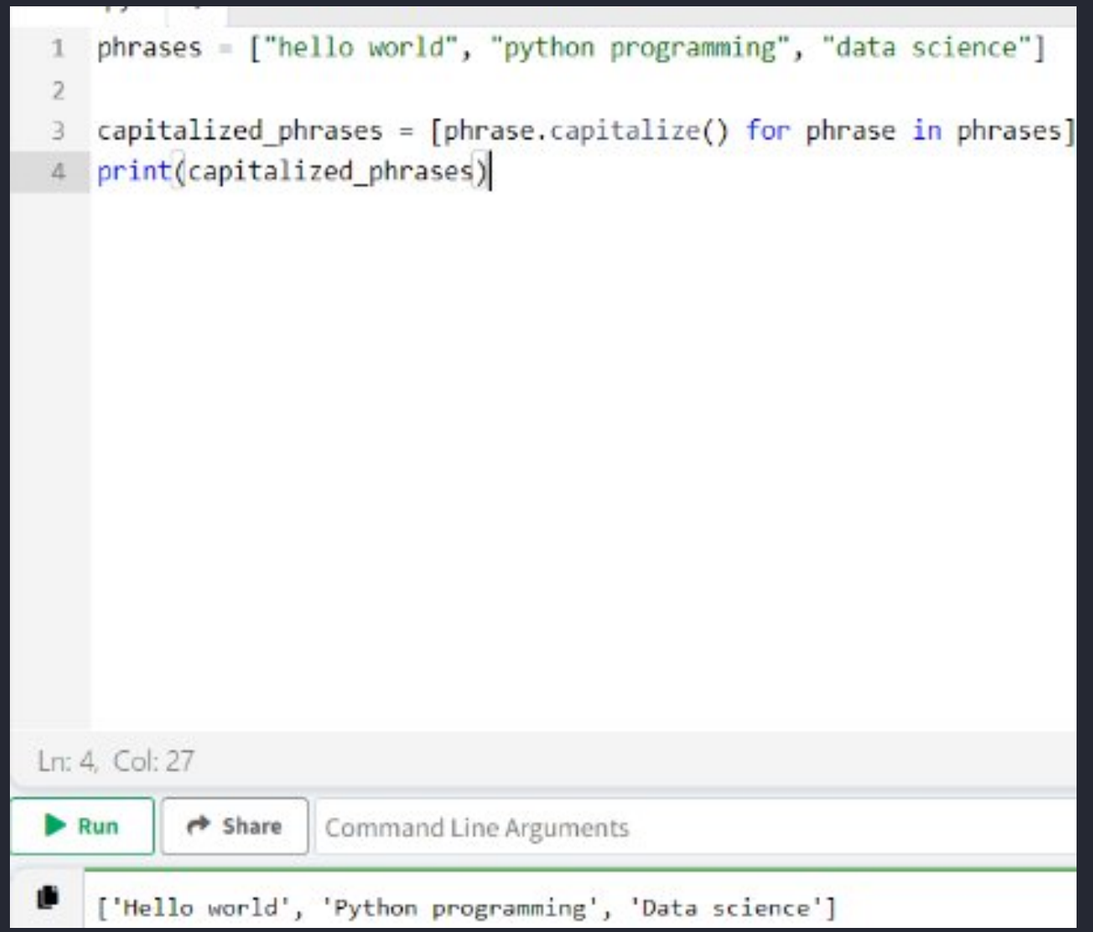


Метод **isalnum()** проверяет, состоит ли строка только из алфавитных символов (букв латинского алфавита, больших и маленьких) и цифр (от 0 до 9), без пробелов, знаков препинания и других специальных символов. Если строка состоит только из букв и цифр, функция возвращает True, иначе False.

Метод **capitalize()**

```
# Приводим к формату
phrases = ["hello world", "python programming", "data science"]

capitalized_phrases = [phrase.capitalize() for phrase in phrases]
print(capitalized_phrases)
```

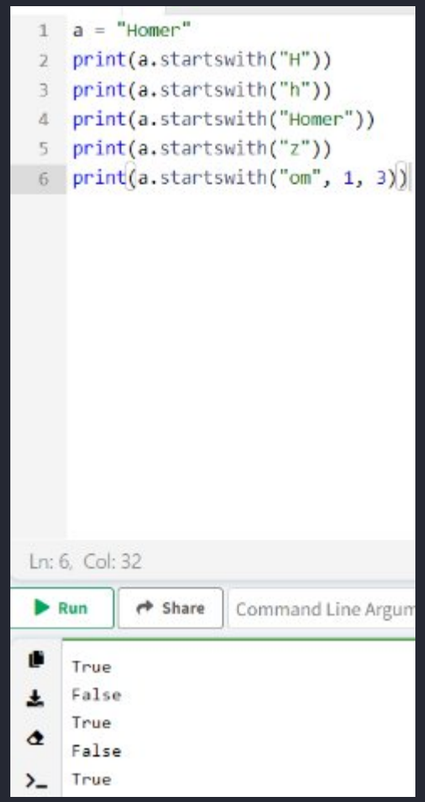


Метод **capitalize()** преобразует первую букву строки в заглавную, а остальные буквы остаются в нижнем регистре.

Метод startswith()

```
a = "Homer"
print(a.startswith("H"))
print(a.startswith("h"))
print(a.startswith("Homer"))
print(a.startswith("z"))
print(a.startswith("om", 1, 3))

#Вывод
True
False
True
False
True
```



Метод **startswith(**prefix[, start[, end]]**)** – возвращает True, если строка начинается с указанного префикса; в противном случае возвращает False.

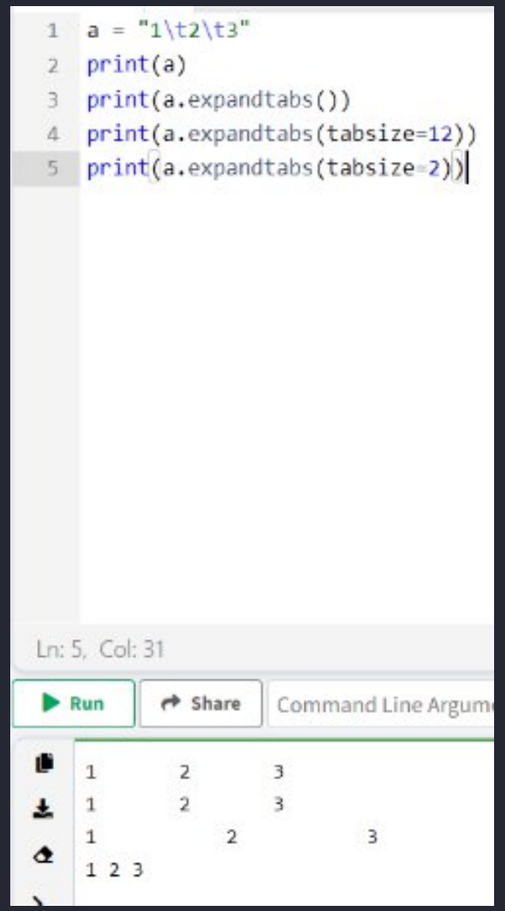
Префикс также может быть кортежем префиксов.

Если указывается аргумент start, проверка начинается с указанной позиции. При указании аргумента end проверка останавливается на указанном символе. Оба аргумента опциональны.

Этот метод — зеркальное отражение метода endswith(), о котором мы уже говорили.

Метод **endswith()**

```
a = "1\t2\t3"
print(a)
print(a.expandtabs())
print(a.expandtabs(tabsize=12))
print(a.expandtabs(tabsize=2))
```



Метод **endswith(**suffix[, start[, end]]**)** – возвращает True, если строка заканчивается на указанный суффикс; в противном случае возвращает False.

Суффикс также может быть кортежем суффиксов.

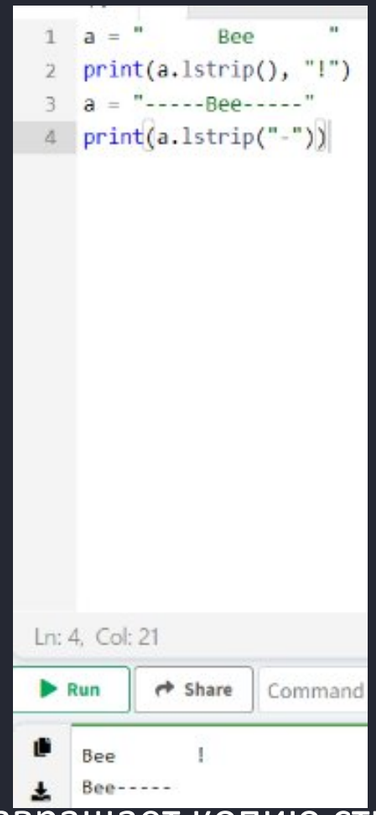
Если указывается аргумент start, проверка начинается с указанной позиции.

При указании аргумента end проверка останавливается на указанном символе. Оба аргумента опциональны.

Метод lstrip()

```
a = "  Bee  "
print(a.lstrip(), "!")
a = "-----Bee-----"
print(a.lstrip("-"))

#Вывод
Bee  !
Bee-----
```



Метод lstrip([chars]) – удаляет указанные или проблемные символы в начале строки. Возвращает копию строки с удаленными первыми символами (т.е., символами, стоящими слева).

В аргументе chars задается набор символов, которые нужно удалить. Если не задавать этот аргумент или установить его в None, по умолчанию будут удалены все пробелы слева.

Обратите внимание, что в качестве аргумента задается не префикс, а именно набор символов, т. е., их порядок не имеет значения. Метод будет удалять все символы из набора, двигаясь слева направо, пока не достигнет первого символа, не входящего в набор.

Метод rstrip()

```
a = "Homer Jay Simpson"
print(a.split())
a = "Homer-Jay-Simpson"
print(a.split(sep="-",maxsplit=1))
a = "Homer,,Bart,"
print(a.split(","))
a = "Homer,,Bart"
print(a.split(",", maxsplit=1))
a = "Homer<>Bart<>Marge"
print(a.split("<>"))
```



Метод rstrip([chars]) – удаляет указанные или проблемные символы в конце строки. Возвращает копию строки с удаленными последними символами (т.е., символами, стоящими справа). В аргументе chars задается набор символов, которые нужно удалить. Если не задавать этот аргумент или установить его в None, по умолчанию будут удалены все пробелы справа.

Обратите внимание, что в качестве аргумента задается не суффикс, а именно набор символов, т. е., их порядок не имеет значения.

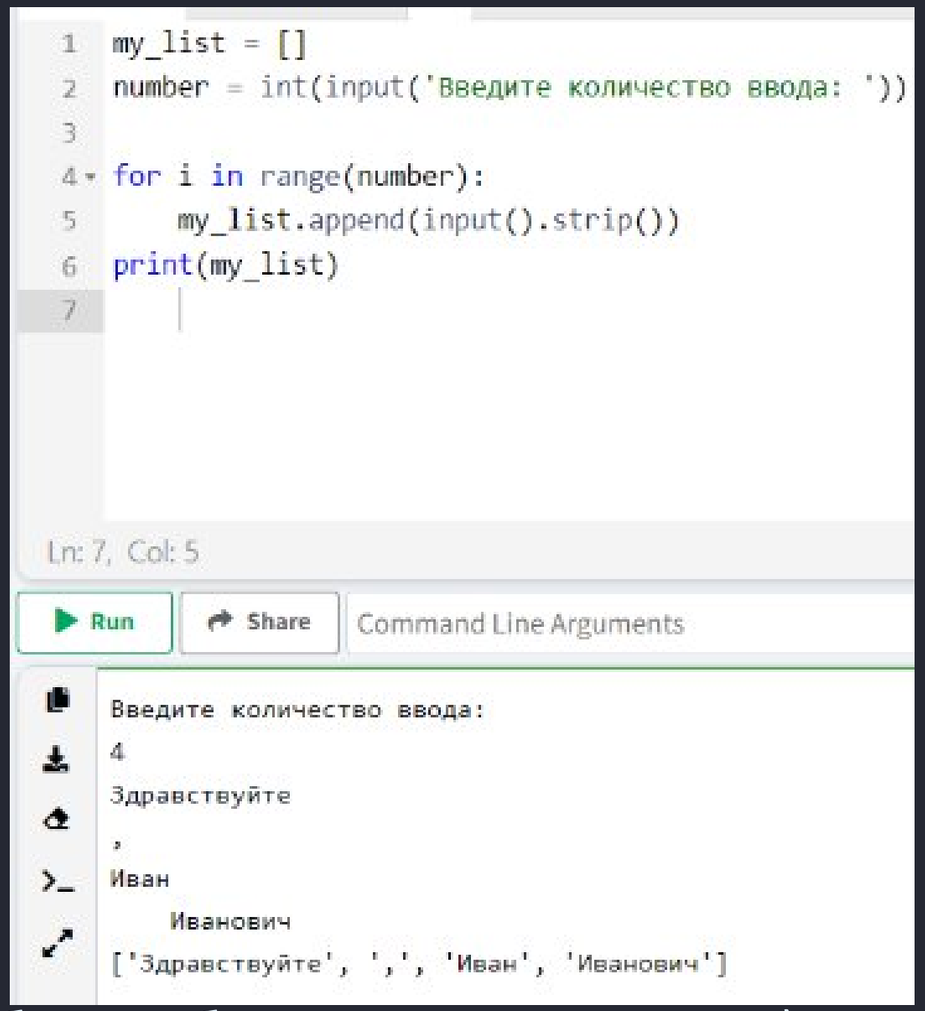
Метод будет удалять все символы из набора, двигаясь справа налево, пока не достигнет первого символа, не входящего в набор.

Работа этого метода — зеркальное отражение работы lstrip().

Метод **strip()** для удаления пробелов

```
my_list = []
number = int(input('Введите количество итераций: '))

for i in range(number):
    my_list.append(input().strip())
print(my_list)
```



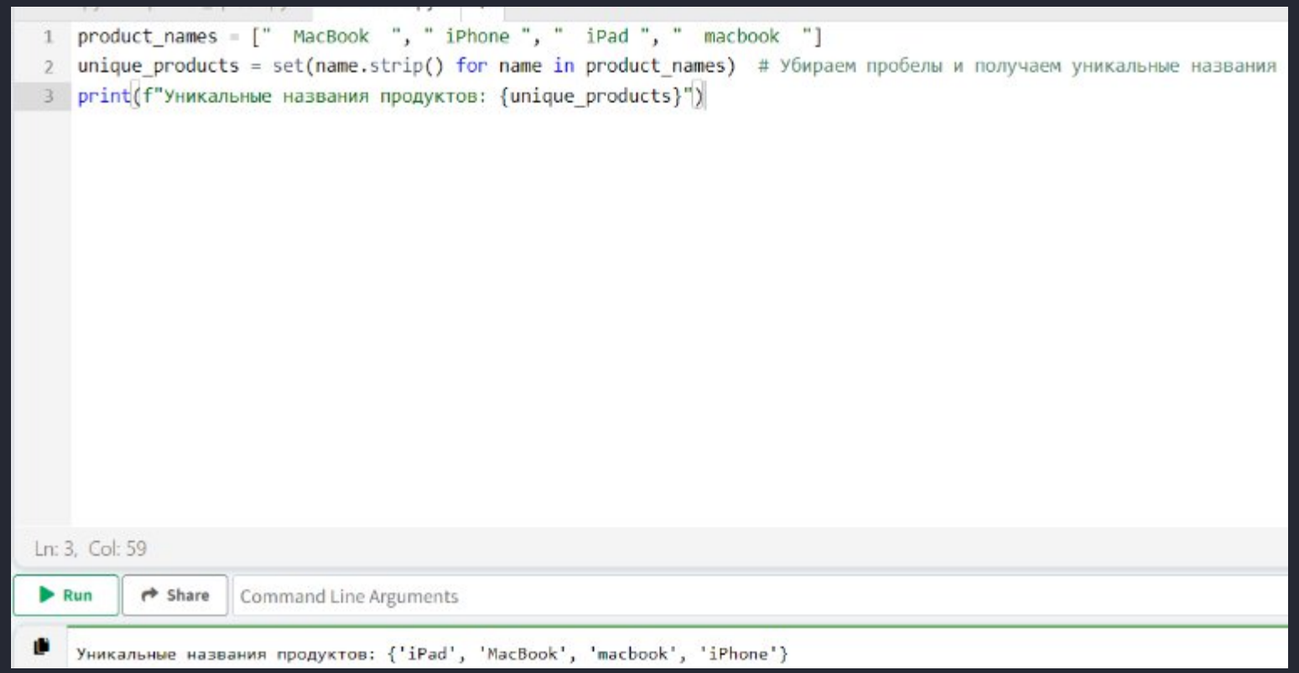
Метод **strip() удаляет пробельные символы** (пробелы, табуляции, переносы строк) **с начала и конца строки**. Это особенно полезно для очистки ввода пользователя от лишних пробелов.

Метод **strip()** для удаления пробелов

#Удалите пробелы в начале и в конце названий продуктов и создайте список уникальных названий.

```
product_names = ["  MacBook  ", " iPhone ", "  iPad ", "  macbook  "]
unique_products = set(name.strip() for name in product_names)
print(f"Уникальные названия продуктов: {unique_products}")

# Вывод
Уникальные названия продуктов: {'iPad', 'MacBook', 'macbook', 'iPhone'}
```



Метод **strip() удаляет пробельные символы** (пробелы, табуляции, переносы строк) **с начала и конца строки**. Это особенно полезно для очистки ввода пользователя от лишних пробелов.